

MANUAL DE INSTALACIÓN · PASO A PASO

SJAP Balance

Manual de instalación paso a paso

Guía para instalar SJAP Balance en un servidor nuevo sin conocimientos de programación. Todo se hace desde el navegador, salvo un paso corto en el que copiarás dos comandos. Tiempo estimado: entre 1 y 2 horas.

ESTACIÓN	VERSIÓN	FECHA	PÚBLICO
EDS LA FLORIDA · Cota	0.1.0	28 de septiembre de 2026	Personal no técnico

01 Antes de empezar

Qué vas a instalar

SJAP Balance tiene dos partes:

PARTE	QUÉ ES	DÓNDE VIVIRÁ
La base de datos	Donde se guardan los cierres, ventas, turnos y usuarios.	Supabase (servicio en la nube)
La página web	Lo que el personal abre en el navegador para usar la app.	Netlify (servicio de publicación web)

Este manual instala la app **vacía**: crea todas las tablas y la configuración básica de la estación, pero **no copia los datos históricos** (cierres, ventas, facturas). Si también hay que trasladar esos datos, pídelo al equipo técnico después de terminar.

Lista de lo que necesitas

- Un computador (Windows o Mac) con Chrome, Edge o Firefox y conexión a internet.
- Un correo electrónico de la empresa para crear las cuentas.
- La **carpeta del proyecto** `sjap-balance` (un archivo `.zip`). Pídelo al responsable técnico; hoy el código está solo en el computador de desarrollo.
- Una contraseña fuerte para el usuario administrador de la app (**master**).
- Un lugar seguro para anotar claves (un gestor de contraseñas o un documento protegido).

CONSEJO

Descomprime el `.zip` en una ubicación fácil de encontrar, por ejemplo el **Escritorio**. Al terminar deberías ver una carpeta llamada `sjap-balance` que contiene, entre otras, las carpetas `src` y `supabase`.

Glosario rápido

PALABRA	QUÉ SIGNIFICA
Proyecto (Supabase)	Tu base de datos en la nube. Cada proyecto tiene su propia dirección.
SQL Editor	Pantalla de Supabase donde se pegan y ejecutan los scripts.
Script	Un archivo de texto con instrucciones para la base de datos. Solo hay que copiarlo y pegarlo.
Clave pública (anon)	Clave que la página web usa para hablar con la base de datos. Es pública, no es secreta.
Clave secreta (service_role)	Clave con acceso total. Nunca la copies en ningún formulario de este manual.
Terminal	Ventana donde se escriben comandos. Solo la usarás en la parte 7.

02 Crear la base de datos en Supabase

1. Entra a <https://supabase.com> y pulsa **Start your project**. Crea la cuenta con el correo de la empresa (o con **Continue with GitHub** si la empresa usa GitHub).
2. Si te pide crear una **organización**, ponle el nombre de la empresa (por ejemplo EDS JAP) y elige el plan. El plan gratuito sirve para probar; para uso real se recomienda el plan **Pro**, que incluye copias de seguridad diarias.
3. Pulsa **New project** y completa:
 - **Name:** `sjap-balance`
 - **Database Password:** pulsa **Generate a password** y **anótala en tu lugar seguro**. No se vuelve a mostrar.
 - **Region:** `South America (São Paulo)`, o `East US` si São Paulo no está disponible.
 - Si aparece una sección **Security options**, deja marcadas las opciones que vienen por defecto (**Data API** activado).
4. Pulsa **Create new project** y espera 1 o 2 minutos hasta que el panel deje de decir *Setting up project*.

NOTA

Supabase cambia a veces los nombres de sus botones. Si un nombre no coincide exactamente con este manual, busca uno parecido en el mismo lugar de la pantalla.

03 Crear las tablas (scripts de Supabase)

En esta parte vas a ejecutar tres scripts que vienen dentro de la carpeta del proyecto, en `sjap-balance/supabase/instalacion/`:

SCRIPT	QUÉ HACE	CUÁNDO
<code>01_crear_tablas.sql</code>	Crea las 32 tablas, las reglas de seguridad y la configuración básica de la estación. No copia datos históricos.	Ahora (paso 3.1)
<code>02_crear_usuario_master.sql</code>	Activa al usuario administrador de la app.	En la parte 4
<code>03_verificar_instalacion.sql</code>	Revisa que todo quedó bien instalado.	En la parte 4

3.1 Ejecutar el script 01 (crear tablas)

1. En la carpeta del proyecto, abre `supabase/instalacion/01_crear_tablas.sql` con el **Bloc de notas** (Windows) o **TextEdit** (Mac): clic derecho → *Abrir con*.
2. Selecciona todo el texto (**Ctrl + A** en Windows, **Cmd + A** en Mac) y cópialo (**Ctrl + C** / **Cmd + C**).
3. En Supabase, en el menú izquierdo, entra a **SQL Editor** y pulsa **New query** (o el botón +).
4. Haz clic dentro del área de texto, pega (**Ctrl + V** / **Cmd + V**) y pulsa **Run** (abajo a la derecha, o **Ctrl + Enter**).
5. Si aparece un aviso sobre *destructive operations* (operaciones destructivas), pulsa **Run this query** para confirmar. Es normal: el script reemplaza unas reglas de seguridad temporales por las definitivas.
6. Espera unos segundos. Debe aparecer abajo **Success. No rows returned**.

IMPORTANTE

Copia el script **desde el archivo**, no desde este PDF. El texto del anexo A es solo de referencia; al copiarlo desde un PDF se pueden colar saltos de línea o caracteres que rompen el script.

CONSEJO

¿Salió un error? No pasa nada: el script es de *todo o nada*, así que si falla no deja la base a medias. Si el error dice «**SJAP Balance ya está instalado en esta base de datos**», las tablas ya existían y no hay que hacer nada más en este paso. Para cualquier otro error, toma una captura de pantalla y envíala al equipo técnico.

04 Activar el usuario administrador (master)

4.1 Crear el usuario

1. En Supabase, entra a **Authentication** → **Users** → **Add user** → **Create new user**.
2. Completa exactamente:
 - **Email:** `master@sjap.local` (así, aunque no sea un correo real; la app lo usa internamente)
 - **Password:** la contraseña del administrador: mínimo 8 caracteres, con letras y números. **Anótala.**
 - Marca la casilla **Auto Confirm User**.
3. Pulsa **Create user**.

4.2 Ejecutar el script 02

Igual que antes: abre `02_crear_usuario_master.sql`, copia todo, y en **SQL Editor** → **New query** pega y pulsa **Run**. El resultado debe mostrar una fila: `master | master | master@sjap.local`.

Contenido del script, como referencia:

```
-- =====
-- SJAP Balance · Script 02 de 03 · ACTIVAR EL USUARIO MASTER
-- =====
-- ANTES de ejecutar este script, crea el usuario en la pantalla de Supabase:
--   Authentication → Users → Add user → Create new user
--   Email:      master@sjap.local
--   Password: la contraseña que usará el administrador de la app
--   Marca la casilla "Auto Confirm User"
--
-- Luego: SQL Editor → New query → pega este archivo → Run.
```

```
-- Resultado esperado: una fila que dice master | master | master@sjap.local
-- Si no aparece ninguna fila, el usuario del paso anterior no existe o el
-- correo quedó mal escrito.
-- =====

insert into public.sjap_usuarios (auth_user_id, estacion_id, username, rol)
select u.id, '884b3769-2c20-4a0b-8019-6972bf5e4caa', 'master', 'master'
from auth.users u
where u.email = 'master@sjap.local'
and not exists (select 1 from public.sjap_usuarios s where s.auth_user_id = u.id);
select s.username as usuario, s.rol, u.email
from public.sjap_usuarios s
join auth.users u on u.id = s.auth_user_id
where s.username = 'master';
```

4.3 Cerrar el registro público

Así nadie puede crearse una cuenta por su cuenta. Los usuarios nuevos los crea el master desde la app.

1. Entra a **Authentication** → **Sign In / Providers** (en algunas versiones: **Authentication** → **Settings**).
2. Desactiva **Allow new users to sign up** y pulsa **Save**.

4.4 Verificar la instalación (script 03)

Abre `03_verificar_instalacion.sql`, cópialo, pégalo en **SQL Editor** → **New query** y pulsa **Run**. Deben aparecer **11 filas, todas con OK** en la columna *estado*:

ELEMENTO	ESPERADO
Tablas de SJAP	32
Reglas de seguridad	124
Funciones	19
Estación	1
Productos	3
Medios de pago	17
Turnos (T1–T4)	4
Esquema de turnos	1
Parámetros de la estación	7
Permisos de la API	32
Usuario master activado	1

Contenido del script, como referencia:

```
-- =====
-- SJAP Balance · Script 03 de 03 · VERIFICAR LA INSTALACIÓN
-- =====
-- SQL Editor → New query → pega este archivo → Run.
-- Resultado esperado: 11 filas y TODAS con "OK" en la columna estado.
-- Si alguna dice "REVISAR", envía una captura de pantalla al equipo técnico.
-- =====

with chequeos(orden, elemento, esperado, encontrado) as (
  values
    (1, 'Tablas de SJAP',          32, (select count(*)::int from pg_tables where schemaname = 'public'
    and tablename like 'sjap\_%')),
    (2, 'Reglas de seguridad',    124, (select count(*)::int from pg_policies where schemaname =
    'public' and tablename like 'sjap\_%')),
    (3, 'Funciones',              19, (select count(*)::int from pg_proc p join pg_namespace n on
    n.oid = p.pronamespace where n.nspname = 'public' and p.proname like 'sjap\_%')),
    (4, 'Estación',               1, (select count(*)::int from public.sjap_estaciones)),
    (5, 'Productos',              3, (select count(*)::int from public.sjap_productos)),
    (6, 'Medios de pago',         17, (select count(*)::int from public.sjap_medios_pago)),
    (7, 'Turnos (T1-T4)',         4, (select count(*)::int from public.sjap_turno_tipos)),
    (8, 'Esquema de turnos',       1, (select count(*)::int from public.sjap_esquemas_turno)),
    (9, 'Parámetros de la estación', 7, (select count(*)::int from public.sjap_estacion_config)),
    (10, 'Permisos de la API',     32, (select count(*)::int from pg_tables where schemaname = 'public'
    and tablename like 'sjap\_%'
                                and has_table_privilege('authenticated',
    format('public.%I', tablename), 'select')
                                and has_table_privilege('authenticated',
    format('public.%I', tablename), 'insert')
                                and has_table_privilege('authenticated',
    format('public.%I', tablename), 'update')
                                and has_table_privilege('authenticated',
    format('public.%I', tablename), 'delete'))),
    (11, 'Usuario master activado', 1, (select count(*)::int from public.sjap_usuarios where rol =
    'master' and activo))
)
select elemento, esperado, encontrado,
       case when encontrado = esperado then 'OK' else 'REVISAR' end as estado
from chequeos
order by orden;
```

ATENCIÓN

Si **Usuario master activado** dice **REVISAR**, revisa que el correo del paso 4.1 sea exactamente `master@sjap.local` y vuelve a ejecutar el script 02. Si otra fila dice **REVISAR**, envía una captura al equipo técnico.

05 Instalar las funciones del servidor

5.1 Función de usuarios (obligatoria)

Con esta función el master crea usuarios, restablece contraseñas, cambia roles y desactiva cuentas desde *Configuración* → *Usuarios*, y cada persona cambia su propia contraseña.

1. En Supabase, entra a **Edge Functions** (menú izquierdo) → **Deploy a new function** → **Via Editor**.
2. En el nombre de la función escribe exactamente: `sjap-usuarios`
3. Borra el código de ejemplo que aparece en el editor.
4. En la carpeta del proyecto, abre `supabase/functions/sjap-usuarios/index.ts` con el Bloc de notas o TextEdit, copia todo y pégalo en el editor de Supabase.

5. Pulsa **Deploy function** y espera el mensaje de éxito.
6. Si la función muestra la opción **Verify JWT** (o *Enforce JWT verification*), déjala **activada**.

5.2 Asistente de chat (opcional)

El botón verde de la app abre un asistente que responde preguntas sobre las cifras. Necesita una clave de Anthropic (servicio de pago por uso). Si no la vas a usar, salta este paso: el resto de la app funciona igual y el botón solo mostrará un error.

1. Repite los pasos de la parte 5.1 con el nombre `chat-asistente` y el archivo `supabase/functions/chat-asistente/index.ts`.
2. En **Edge Functions** → **Secrets** (o *Manage secrets*), agrega un secreto llamado `ANTHROPIC_API_KEY` con la clave que te entregue el equipo técnico, y pulsa **Save**.

06 Copiar los dos datos de conexión

La página web necesita saber a qué base de datos conectarse. Busca estos dos datos y pégalos temporalmente en tu documento seguro:

1. **Project URL:** en Supabase, **Project Settings** (engranaje, abajo a la izquierda) → **Data API** (o **API**). Tiene la forma `https://abcdefghijklmnop.supabase.co`.
2. **Clave pública:** en **Project Settings** → **API Keys**. Copia la clave marcada como **anon public** o **publishable** (empieza por `eyJ` o por `sb_publishable_`).

CUIDADO

No copies la clave `service_role` ni ninguna marcada como **secret**. Si por error la pegas en el paso siguiente, el asistente la rechazará.

07 Preparar la página web

Este es el único paso con comandos: solo hay que copiar y pegar.

7.1 Instalar Node.js (una sola vez)

1. Entra a <https://nodejs.org> y descarga la versión marcada como **LTS**.
2. Abre el instalador y pulsa **Siguiente** / **Continuar** hasta terminar, aceptando las opciones por defecto.

7.2 Abrir la terminal dentro de la carpeta del proyecto

- **Windows:** abre la carpeta `sjap-balance` en el Explorador de archivos, haz clic en la barra de dirección (arriba), escribe `cmd` y pulsa **Enter**. Se abre una ventana negra.
- **Mac:** abre la aplicación **Terminal**, escribe `cd` (con un espacio al final), **arrastra la carpeta** `sjap-balance` a la ventana y pulsa **Enter**.

7.3 Ejecutar los dos comandos

Escribe (o pega) este comando y pulsa **Enter**. Descarga los componentes de la app y tarda 1 o 2 minutos:

```
npm install
```

Luego este otro, y pulsa **Enter**:

```
npm run preparar
```

El asistente te hará dos preguntas. Pega cada dato de la parte 6 y pulsa **Enter**:

```
SJAP Balance · Preparar la app para publicar

1) Project URL: https://abcdefghijklmnop.supabase.co
2) Clave pública (anon / publishable): eyJhbGciOi...

Compilando... (tarda menos de un minuto)

✓ Listo. La app quedó en la carpeta:
  ../sjap-balance/dist
```

Al terminar aparece una carpeta nueva llamada `dist` dentro de `sjap-balance`. Esa carpeta **es la página web** lista para publicar.

CONSEJO

Si escribes mal un dato, el asistente te lo dice y te lo vuelve a pedir. Para pegar en la ventana de Windows usa **clic derecho**; en Mac, **Cmd + V**.

08 Publicar la página web en Netlify

1. Entra a <https://app.netlify.com> y crea una cuenta gratuita con el correo de la empresa.
2. En el panel, pulsa **Add new site** (o **Add new project**) → **Deploy manually**.
3. **Arrastra la carpeta** `dist` (la del paso 7) al recuadro que dice *Drag and drop your site output folder here*.
4. Espera unos segundos. Netlify te dará una dirección como `https://nombre-al-azar-123.netlify.app`.
5. Para cambiar esa dirección por una más clara: **Site configuration** → **Change site name** → por ejemplo `sjap-eds-la-florida`. La dirección quedará `https://sjap-eds-la-florida.netlify.app`.

NOTA

Si la empresa tiene su propio servidor web o dominio (por ejemplo `sjap.miempresa.com`), entrega la carpeta `dist` al área de sistemas junto con el documento técnico **SJAP-Balance-Despliegue.pdf**, que explica cómo configurarlo.

09 Primer ingreso y pruebas

- Abre la dirección de Netlify. Debe aparecer la pantalla **SJAP Balance** con los campos *Usuario* y *Contraseña*.
- Entra con usuario `master` y la contraseña del paso 4.1.
- Ve a **Configuración** → **General** y completa los datos de la estación (NIT, dirección, teléfono). Pulsa **Guardar**.
- En **Configuración** → **Usuarios**, crea un usuario para cada persona del equipo (rol **Dependiente** para la operación diaria). Usa **Generar** para la contraseña temporal y entrégasela; la app le pedirá cambiarla al entrar por

primera vez.

- Entra a **Cargar** y sube el archivo del cierre de un día. Debe terminar en **completado**.
- Recarga la página con **F5** estando en **Diarios**. Debe seguir mostrando la app (no un error 404).

Si todo lo anterior funciona, **la instalación está terminada**. Comparte la dirección con el personal junto con el **Manual de usuario**.

10 Si algo sale mal

QUÉ VES	QUÉ HACER
El script 01 dice «ya está instalado»	Las tablas ya existen; sigue con la parte 4.
El script 01 muestra otro error	No se aplicó nada. Envía una captura del mensaje al equipo técnico.
El script 03 dice «REVISAR» en <i>Usuario master</i>	Revisa el correo <code>master@sjap.local</code> (parte 4.1) y repite el script 02.
<code>npm</code> no se reconoce como comando	Node.js no quedó instalado: repite la parte 7.1 y cierra y vuelve a abrir la terminal.
El asistente rechaza la clave	Estás copiando la clave secreta. Copia la anon public / publishable (parte 6).
La página abre pero «Usuario o contraseña incorrectos»	Revisa la contraseña. Si una persona olvidó la suya, el master le asigna una temporal en Configuración → Usuarios → Restablecer contraseña . Si el que la olvidó es el único master, el equipo técnico debe asignarle una nueva desde Supabase (la recuperación por correo no funciona: el correo <code>@sjap.local</code> no existe).
«Este usuario está desactivado»	Un master lo desactivó. Puede reactivarlo en Configuración → Usuarios .
«Se cerró la sesión por inactividad»	Normal: tras 30 minutos sin uso la sesión se cierra por seguridad. Vuelve a entrar.
Error 404 al recargar una página	La carpeta publicada no fue <code>dist</code> , o se publicó su contenido incompleto. Vuelve a arrastrar la carpeta <code>dist</code> completa.
El botón verde del asistente da error	Es normal si no instalaste el asistente (parte 5.2) o falta el secreto <code>ANTHROPIC_API_KEY</code> .
Olvidaste la contraseña de la base de datos	Supabase → Project Settings → Database → <i>Reset database password</i> .

Qué guardar al terminar

- La contraseña de la base de datos (parte 2).
- La contraseña del usuario master (parte 4).
- La dirección de la app en Netlify (parte 8).
- Las cuentas de Supabase y Netlify, con el correo con el que se crearon.
- La carpeta del proyecto `sjap-balance` (para futuras actualizaciones).

Anexo A. Script 01 — crear tablas (referencia)

Este es el contenido completo de `supabase/instalacion/01_crear_tablas.sql`. **Para instalar, cópialo desde el archivo, no desde aquí** (ver parte 3.1). Se incluye para que el equipo técnico pueda revisarlo. En la versión PDF, las líneas muy largas aparecen partidas en dos: la continuación se marca con `\`.

```
-- =====
-- SJAP Balance · Script 01 de 03 · CREAR TABLAS
-- =====
-- Qué hace: crea la estructura completa de la base de datos de SJAP Balance
-- (32 tablas, reglas de seguridad y funciones) en un proyecto Supabase NUEVO.
--
-- NO copia datos históricos (cierres, ventas, facturas...). Solo deja la
-- configuración mínima para arrancar: la estación EDS LA FLORIDA, sus 3
-- productos, 17 medios de pago, los turnos T1-T4 y los parámetros de alertas.
--
-- Cómo usarlo: Supabase → SQL Editor → New query → pegar TODO este archivo →
-- Run. Resultado esperado: "Success. No rows returned".
--
-- Si algo falla, no se aplica nada (todo o nada). Si la base ya tenía SJAP
-- instalado, el script se detiene sin tocar nada.
--
-- Generado el 2026-09-28 a partir de supabase/migrations/ (26 archivos, en orden)
-- con: npm run generar:instalacion - no editar a mano.
-- =====
do $$
begin
  if to_regclass('public.sjap_estaciones') is not null then
    raise exception 'SJAP Balance ya está instalado en esta base de datos (existe la tabla
\ sjap_estaciones). No se hizo ningún cambio.';
  end if;
end $$;

-- =====
-- 20260901151847_sjap_balance_schema_core.sql
-- =====
--
-- SJAP – Balance diario/mensual de estaciones de servicio (multi-EDS)
-- Prefijo sjap_ para aislar de las demás tablas de la cuenta.
-- =====

create extension if not exists pgcrypto;

-- =====
-- 1. Estaciones (multi-EDS desde el día 1)
-- =====

create table public.sjap_estaciones (
  id uuid primary key default gen_random_uuid(),
  nombre text not null,
  razon_social text,
  nit text,
  ciudad text,
  bandera text default 'Terpel',
  activa boolean not null default true,
```

```

    created_at timestamptz not null default now(),
    unique (nombre)
);
comment on table public.sjap_estaciones is 'Cada estación de servicio (EDS) gestionada en la app. Todo
↳ el resto del modelo cuelga de aquí.';

-----
-- 2. Catálogo de productos por estación (NO es un enum fijo: cada EDS puede
-- nombrar sus productos distinto – ej. "Bioacem" vs "ACPM" – sin confirmar aún).
-----
create table public.sjap_productos (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    codigo text not null,          -- tal como aparece en los archivos (p.ej.
    ↳ 'CORRIENTE', 'BIOACEM', 'EXTRA')
    nombre_visible text not null,   -- nombre a mostrar en UI (editable, p.ej. 'ACPM / Diésel')
    unidad text not null default 'GALONES',
    orden smallint not null default 0,
    activo boolean not null default true,
    created_at timestamptz not null default now(),
    unique (estacion_id, codigo)
);
comment on table public.sjap_productos is 'Catálogo de combustibles por estación. codigo debe calzar con
↳ el texto que trae el archivo diario (PRODUCTO / DESPACHOS PRODUCTOS).';

-- Historial de precios por producto (el precio cambia a mitad de mes en los
-- archivos reales – cada movimiento debe valorarse al precio vigente en su fecha).
create table public.sjap_precios_producto (
    id uuid primary key default gen_random_uuid(),
    producto_id uuid not null references public.sjap_productos(id) on delete cascade,
    precio numeric(14,2) not null,
    vigente_desde date not null,
    created_at timestamptz not null default now(),
    unique (producto_id, vigente_desde)
);
comment on table public.sjap_precios_producto is 'Historial de precio por producto. El precio vigente
↳ para una fecha es el de mayor vigente_desde <= fecha.';
create index idx_sjap_precios_producto_lookup on public.sjap_precios_producto (producto_id,
↳ vigente_desde desc);

-----
-- 3. Archivos de cierre diario cargados (auditoría / trazabilidad de origen)
-----
create table public.sjap_archivos_cierre (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    fecha date not null,
    nombre_archivo text not null,
    tamano_bytes bigint,
    hash_archivo text,             -- para detectar recargas del mismo archivo
    estado text not null default 'recibido'
    check (estado in ('recibido', 'procesando', 'procesado', 'con_errores', 'error')),
    hojas_detectadas jsonb,        -- snapshot de qué hojas/tablas encontró el parser
    errores jsonb,
    version int not null default 1, -- se incrementa en cada reprocesamiento
    subido_por text,
    procesado_en timestamptz,
    created_at timestamptz not null default now(),
    unique (estacion_id, fecha)
);

```

```

comment on table public.sjap_archivos_cierre is 'Un registro por (estación, fecha): el archivo diario
↳ vigente para ese día. Reprocesar incrementa version y re-dispara la extracción.';

create table public.sjap_archivos_cierre_historial (
  id uuid primary key default gen_random_uuid(),
  archivo_id uuid not null references public.sjap_archivos_cierre(id) on delete cascade,
  version int not null,
  hash_archivo text,
  estado_resultado text not null,
  errores jsonb,
  procesado_en timestamptz not null default now()
);
comment on table public.sjap_archivos_cierre_historial is 'Historial completo de cada intento de
↳ procesamiento de un archivo (incluye reprocesos), para auditoría.';

-----
-- 4. Transacciones crudas extraídas (nivel más fino: 1 fila por venta)
-----

create table public.sjap_transacciones (
  id uuid primary key default gen_random_uuid(),
  archivo_id uuid not null references public.sjap_archivos_cierre(id) on delete cascade,
  estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
  categoria text not null check (categoria in ('COMBUSTIBLE','CANASTILLA','KIOSCO','CDL','COMPLEMENTARIOS
↳ ')),
  consecutivo text,
  prefijo text,
  fecha date not null,
  hora time,
  promotor_nombre text,
  tipo_factura text,                -- 'ELECTRONICA','CREDITO', etc. (abierto, no enum)
  producto_nombre text,
  cantidad numeric(14,3),
  unidad text,
  subtotal numeric(14,2),
  descuento numeric(14,2) default 0,
  impuesto_total numeric(14,2) default 0,
  total numeric(14,2),
  created_at timestamptz not null default now()
);
comment on table public.sjap_transacciones is 'Detalle transaccional extraído de las tablas de venta
↳ item-a-item del archivo diario (COMBUSTIBLE, CANASTILLA, KIOSCO, CDL, VENTAS COMPLEMENTARIOS). Base
↳ para trazabilidad total.';
create index idx_sjap_transacciones_estacion_fecha on public.sjap_transacciones (estacion_id, fecha);
create index idx_sjap_transacciones_archivo on public.sjap_transacciones (archivo_id);

-- Resumen "VENTAS POR PROMOTOR" que ya viene calculado dentro del archivo diario
create table public.sjap_ventas_promotor_resumen (
  id uuid primary key default gen_random_uuid(),
  archivo_id uuid not null references public.sjap_archivos_cierre(id) on delete cascade,
  estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
  fecha date not null,
  promotor_nombre text not null,
  numero_ventas int not null default 0,
  total_ventas numeric(14,2) not null default 0,
  created_at timestamptz not null default now(),
  unique (archivo_id, promotor_nombre)
);
comment on table public.sjap_ventas_promotor_resumen is 'Bloque "VENTAS POR PROMOTOR" tal cual viene en
↳ el archivo diario (agregado, sin turno/isla).';

```

```

-- Resumen "VENTAS MEDIO DE PAGO" que ya viene calculado dentro del archivo diario
create table public.sjap_ventas_medio_pago (
  id uuid primary key default gen_random_uuid(),
  archivo_id uuid not null references public.sjap_archivos_cierre(id) on delete cascade,
  estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
  fecha date not null,
  medio_pago text not null,          -- texto libre: 'EFECTIVO','RUMBO','DATAFONO','QR','VIVE TERPEL',
  ↳ etc.
  numero_ventas int not null default 0,
  total_ventas numeric(14,2) not null default 0,
  created_at timestamptz not null default now(),
  unique (archivo_id, medio_pago)
);
comment on table public.sjap_ventas_medio_pago is 'Bloque "VENTAS MEDIO DE PAGO" tal cual viene en el
  ↳ archivo diario. medio_pago es texto libre porque cada EDS puede tener medios distintos.';

-- Resumen "DESPACHOS PRODUCTOS" (galones por producto) – insumo clave del inventario
create table public.sjap_despachos_producto (
  id uuid primary key default gen_random_uuid(),
  archivo_id uuid not null references public.sjap_archivos_cierre(id) on delete cascade,
  estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
  producto_id uuid references public.sjap_productos(id),
  producto_nombre_original text not null, -- texto tal cual venía en el archivo (por si no calzó con el
  ↳ catálogo)
  fecha date not null,
  unidad text,
  cantidad numeric(14,3) not null default 0,
  precio numeric(14,2),
  descuento numeric(14,2) default 0,
  venta_total numeric(14,2),
  created_at timestamptz not null default now(),
  unique (archivo_id, producto_nombre_original)
);
comment on table public.sjap_despachos_producto is 'Bloque "DESPACHOS PRODUCTOS": ventas en galones por
  ↳ producto, tal cual viene en el archivo. Alimenta sjap_balance_diario.ventas_galones.';
;

-----
-- 20260901151934_sjap_balance_schema_manual_capture_and_balances.sql
-----
-- =====
-- SJAP – Insumos manuales, balance calculado, cuentas de clientes, auditoría
-- =====

-----
-- 5. Insumos manuales – NO vienen en el archivo diario. Se deja el modelo
-- diseñado a la espera de definir con el cliente cómo se capturan
-- (formulario en la app vs. otro archivo). Todos son nullable/editable
-- y quedan vinculados a estación+producto+fecha o estación+fecha.
-----

create table public.sjap_lecturas_inventario (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
  producto_id uuid not null references public.sjap_productos(id) on delete cascade,
  fecha date not null,
  inventario_final_real numeric(14,3) not null, -- lectura física con vara de tanque
  origen text not null default 'manual' check (origen in ('manual','importado')),
  capturado_por text,
  capturado_en timestamptz not null default now(),

```

```

    notas text,
    unique (estacion_id, producto_id, fecha)
);
comment on table public.sjap_lecturas_inventario is 'Lectura física de inventario (vara de tanque) al
↳ cierre del día. Insumo manual – pendiente de definir flujo de captura exacto con el cliente.';

create table public.sjap_facturas_compra (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    producto_id uuid not null references public.sjap_productos(id) on delete cascade,
    fecha date not null,
    numero_factura text,
    cantidad numeric(14,3) not null,
    origen text not null default 'manual' check (origen in ('manual','importado')),
    capturado_por text,
    capturado_en timestampz not null default now(),
    notas text
);
comment on table public.sjap_facturas_compra is 'Facturas de compra/recibo de combustible del proveedor.
↳ Insumo manual – pendiente de definir si se digita, se sube un PDF/foto, o llega en otro archivo.';
create index idx_sjap_facturas_compra_estacion_fecha on public.sjap_facturas_compra (estacion_id, fecha);

create table public.sjap_efectivo_diario (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    fecha date not null,
    efectivo_real numeric(14,2),          -- efectivo contado físicamente
    certificacion_bancaria numeric(14,2), -- consignación certificada por el banco (esporádica, no todos
↳ los días)
    origen text not null default 'manual' check (origen in ('manual','importado')),
    capturado_por text,
    capturado_en timestampz not null default now(),
    notas text,
    unique (estacion_id, fecha)
);
comment on table public.sjap_efectivo_diario is 'Efectivo real contado y certificación bancaria del día.
↳ Insumo manual – determina el descuadre de caja (Diferencia) en sjap_cierre_diario.';

-- -----
-- 6. Cuentas de clientes propios (registro abierto – se desconoce el origen
--   de los datos día a día; se deja el modelo listo para cargarlos manual
--   o por archivo cuando se aclare con el cliente).
-- -----

create table public.sjap_cuentas_cliente (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    nombre text not null,
    nit text,
    activo boolean not null default true,
    created_at timestampz not null default now(),
    unique (estacion_id, nombre)
);
comment on table public.sjap_cuentas_cliente is 'Cuentas corporativas de crédito ("clientes propios"):
↳ CRC, EASY CEM, COLEG NUEVA INGLATERRA, etc. Lista abierta y creciente, no un enum.';

create table public.sjap_movimientos_cuenta_cliente (
    id uuid primary key default gen_random_uuid(),
    cuenta_id uuid not null references public.sjap_cuentas_cliente(id) on delete cascade,
    archivo_id uuid references public.sjap_archivos_cierre(id) on delete set null,
    fecha date not null,

```

```

    tipo text not null check (tipo in ('COMBUSTIBLE','UREA','LUBRICANTES')),
    monto numeric(14,2) not null default 0,
    origen text not null default 'manual' check (origen in ('manual','importado')),
    created_at timestamptz not null default now()
);
comment on table public.sjap_movimientos_cuenta_cliente is 'Consumo diario por cuenta de cliente propio,
↳ separado por tipo. origen=importado cuando se defina la fuente real.';
create index idx_sjap_mov_cuenta_cliente_fecha on public.sjap_movimientos_cuenta_cliente (cuenta_id,
↳ fecha);

-----
-- 7. Venta por promotor con turno/isla – fuente desconocida hoy (no viene en
--    el archivo diario). Se deja el modelo con "origen" para diferenciar
--    datos derivados automáticamente (agrupando por hora) de datos reales.
-----

create table public.sjap_ventas_promotor_turno (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    fecha date not null,
    promotor_nombre text not null,
    turno text check (turno in ('T1','T2','T3','T4')),
    isla text,
    galones numeric(14,3),
    numero_clientes int,
    origen text not null default 'derivado' check (origen in ('manual','derivado','importado')),
    created_at timestamptz not null default now()
);
comment on table public.sjap_ventas_promotor_turno is 'Detalle de venta por promotor/turno/isla en
↳ galones. origen=derivado cuando se infiere agrupando transacciones por hora; ajustar cuando se
↳ conozca la fuente real.';
create index idx_sjap_ventas_promotor_turno_fecha on public.sjap_ventas_promotor_turno (estacion_id,
↳ fecha);

-----
-- 8. Balance diario calculado por producto (espeja G. Corriente / Bioacem / G. Extra)
-----

create table public.sjap_balance_diario_producto (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    producto_id uuid not null references public.sjap_productos(id) on delete cascade,
    fecha date not null,
    inventario_inicial numeric(14,3),
    ventas_galones numeric(14,3) not null default 0,
    recibos_galones numeric(14,3) not null default 0,
    inventario_teorico numeric(14,3),
    inventario_final_real numeric(14,3),      -- copiado de sjap_lecturas_inventario cuando exista
    fluctuacion_dia numeric(14,3),
    fluctuacion_acumulada numeric(14,3),
    precio_vigente numeric(14,2),
    fluctuacion_valor numeric(14,2),
    estado text not null default 'pendiente_insumos'
    check (estado in ('pendiente_insumos','completo')),
    calculado_en timestamptz not null default now(),
    unique (estacion_id, producto_id, fecha)
);
comment on table public.sjap_balance_diario_producto is 'Reconciliación diaria de inventario por
↳ producto. inventario_inicial = inventario_final_real del día anterior (o teórico si aún no hay
↳ lectura real). Se recalcula cuando cambian sus insumos.';
create index idx_sjap_balance_diario_producto_fecha on public.sjap_balance_diario_producto (estacion_id,
↳ fecha);

```

```

-- -----
-- 9. Cierre diario consolidado (espeja la hoja "Cierre": 1 fila por día)
-- -----
create table public.sjap_cierre_diario (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    archivo_id uuid references public.sjap_archivos_cierre(id) on delete set null,
    fecha date not null,
    venta_total numeric(14,2) not null default 0,
    venta_galones_total numeric(14,3) not null default 0,
    numero_clientes int,
    efectivo_calculado numeric(14,2), -- residual: total - suma(otros medios de pago)
    efectivo_real numeric(14,2), -- de sjap_efectivo_diario
    diferencia_caja numeric(14,2), -- efectivo_calculado - efectivo_real
    diferencia_caja_acumulada numeric(14,2),
    certificacion_bancaria numeric(14,2),
    diferencia_certificacion numeric(14,2),
    estado text not null default 'pendiente_insumos'
    check (estado in ('pendiente_insumos','completo','con_alertas')),
    calculado_en timestamptz not null default now(),
    unique (estacion_id, fecha)
);
comment on table public.sjap_cierre_diario is 'Balance diario consolidado (equivalente a la hoja
↳ "Cierre" del Excel). Los medios de pago viven en sjap_ventas_medio_pago en vez de columnas fijas.';

-- -----
-- 10. Auditoría / historial de procesamiento
-- -----
create table public.sjap_auditoria (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid references public.sjap_estaciones(id) on delete cascade,
    archivo_id uuid references public.sjap_archivos_cierre(id) on delete cascade,
    entidad text not null,
    entidad_id uuid,
    accion text not null,
    nivel text not null default 'info' check (nivel in ('info','warning','error')),
    detalle jsonb,
    created_by text,
    created_at timestamptz not null default now()
);
comment on table public.sjap_auditoria is 'Bitácora de procesamiento: qué se extrajo, qué falló, qué se
↳ recalculó y por qué.';
create index idx_sjap_auditoria_archivo on public.sjap_auditoria (archivo_id);
;

-- -----
-- 20260901151948_sjap_balance_schema_rls.sql
-- -----
-- =====
-- SJAP – RLS: habilitado en todas las tablas, política permisiva para
-- usuarios autenticados mientras se define el modelo de acceso real
-- (multi-EDS con permisos por estación queda pendiente de diseño).
-- =====

do $$
declare
    t text;
begin
    for t in

```

```

select tablename from pg_tables
where schemaname = 'public' and tablename like 'sjap_%'
loop
execute format('alter table public.%I enable row level security', t);
execute format(
    'create policy %I on public.%I for all to authenticated using (true) with check (true)',
    t || '_authenticated_all', t
);
end loop;
end $$;
;

-----
-- 20260901153813_sjap_allow_anon_for_poc.sql
-----

-- POC: la app aún no tiene login propio, así que se permite también al rol
-- anon (usa la publishable key). Antes de exponer esto a más usuarios que
-- Iván/Javier hay que: (1) agregar autenticación real y (2) restringir estas
-- políticas a `authenticated` con reglas por estación.
do $$
declare
    t text;
begin
    for t in
        select tablename from pg_tables
        where schemaname = 'public' and tablename like 'sjap_%'
    loop
        execute format(
            'create policy %I on public.%I for all to anon using (true) with check (true)',
            t || '_anon_poc', t
        );
    end loop;
end $$;
;

-----
-- 20260901153818_sjap_productos_alias.sql
-----

alter table public.sjap_productos add column if not exists alias text[] not null default '{}';
comment on column public.sjap_productos.alias is 'Otros nombres con los que este producto puede aparecer
↳ en los archivos (ej. BIOACEM tiene alias {DIESEL}). El parser resuelve producto_id contra codigo o
↳ cualquier alias.';

update public.sjap_productos set alias = array['DIESEL'] where codigo = 'BIOACEM';
;

-----
-- 20260901153900_sjap_seed_estacion_base.sql
-----

-- Semilla base de la estación, para poder levantar un entorno NUEVO desde cero
-- con `supabase db push` / `supabase db reset`.
--
-- Agregada el 2026-09-23. En el proyecto original estos registros se crearon a
-- mano (fuera de migraciones) el 2026-09-01; este archivo los reproduce con los
-- MISMOS ids porque la migración 20260910174944 referencia el id de la estación
-- de forma fija. Es idempotente: en una base que ya tiene los datos no hace nada.
--
-- Para otra estación distinta a EDS LA FLORIDA: cambia nombre/ciudad aquí, pero
-- conserva el id (o ajusta también 20260910174944 antes de aplicar).
```

```

insert into public.sjap_estaciones (id, nombre, razon_social, ciudad, bandera, activa)
values ('884b3769-2c20-4a0b-8019-6972bf5e4caa', 'EDS LA FLORIDA', 'EDS JAP S.A.S.', 'Cota', 'Terpel',
    ↳ true)
on conflict (id) do nothing;

insert into public.sjap_productos (id, estacion_id, codigo, nombre_visible, unidad, orden, activo, alias)
values
    ('84a8ba37-2e42-46a3-9d7c-b0d4edebfa79', '884b3769-2c20-4a0b-8019-6972bf5e4caa', 'CORRIENTE',
    ↳ 'Corriente', 'GALONES', 1, true, '{}'),
    ('6e896414-4056-4171-89ca-5320b71776a7', '884b3769-2c20-4a0b-8019-6972bf5e4caa', 'BIOACEM', 'ACPM /
    ↳ Diésel (Bioacem)', 'GALONES', 2, true, array['DIESEL']),
    ('092f4120-f868-4d4e-9d5b-470436512899', '884b3769-2c20-4a0b-8019-6972bf5e4caa', 'EXTRA', 'Extra',
    ↳ 'GALONES', 3, true, '{}')
on conflict (estacion_id, codigo) do nothing;

-- Precios históricos usados por los cierres de ene-feb 2026. En un entorno
-- nuevo, agrega aquí (o por SQL) los precios vigentes reales.
insert into public.sjap_precios_producto (producto_id, precio, vigente_desde)
values
    ('84a8ba37-2e42-46a3-9d7c-b0d4edebfa79', 15590, '2026-02-01'),
    ('84a8ba37-2e42-46a3-9d7c-b0d4edebfa79', 15190, '2026-02-05'),
    ('6e896414-4056-4171-89ca-5320b71776a7', 10750, '2026-01-21'),
    ('092f4120-f868-4d4e-9d5b-470436512899', 19770, '2026-01-01'),
    ('092f4120-f868-4d4e-9d5b-470436512899', 18070, '2026-02-07')
on conflict (producto_id, vigente_desde) do nothing;
;

-----
-- 20260901173017_sjap_balance_schema_urea_presupuesto_sicom.sql
-----
=====
-- SJAP – Hojas adicionales del balance mensual: Urea, Presupuesto, Sicom.
-- Se agregan tal como existen en el Excel de referencia, sin inventar
-- reglas de negocio para lo que aún no está confirmado con el cliente.
-----

create table public.sjap_urea_diario (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    fecha date not null,
    inventario_inicial numeric(14,3),
    inventario_final numeric(14,3),
    inventario_teorico numeric(14,3),
    diferencia numeric(14,3),
    recibo numeric(14,3),
    rumbo_litros numeric(14,3),
    clientes_propios_litros numeric(14,3),
    valor numeric(14,2),
    valor_rumbo numeric(14,2),
    valor_propios numeric(14,2),
    total numeric(14,2),
    costo_unitario numeric(14,2),
    origen text not null default 'importado' check (origen in ('manual','importado')),
    created_at timestamptz not null default now(),
    unique (estacion_id, fecha)
);
comment on table public.sjap_urea_diario is 'Espeja la hoja "Urea": inventario/recibo/valor diario de
    ↳ urea (AdBlue), paralelo al de combustible.';

create table public.sjap_presupuesto_mensual (

```

```

    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    anio int not null,
    mes int not null check (mes between 1 and 12),
    ventas numeric(14,2),
    compra numeric(14,2),
    rumbo numeric(14,2),
    clientes_propios numeric(14,2),
    clientes_paso numeric(14,2),
    presupuesto_total numeric(14,2),
    presupuesto_corriente numeric(14,2),
    presupuesto_acpm numeric(14,2),
    presupuesto_extra numeric(14,2),
    cumplimiento numeric(8,4),
    pct_rumbo numeric(8,4),
    pct_clientes_propios numeric(8,4),
    pct_clientes_paso numeric(8,4),
    origen text not null default 'importado' check (origen in ('manual','importado')),
    created_at timestamptz not null default now(),
    unique (estacion_id, anio, mes)
);
comment on table public.sjap_presupuesto_mensual is 'Espeja la hoja "Presupuesto": una fila por mes con
↳ ventas/compra reales vs. presupuestadas por producto.';

create table public.sjap_sicom_mensual (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references public.sjap_estaciones(id) on delete cascade,
    producto_id uuid references public.sjap_productos(id),
    producto_nombre_original text not null,
    anio int not null,
    mes int not null check (mes between 1 and 12),
    inventario_inicial numeric(14,3),
    compras numeric(14,3),
    ventas numeric(14,3),
    faltantes numeric(14,3),
    evaporacion numeric(14,3),
    inventario_final_calculado numeric(14,3),
    inventario_final_real numeric(14,3),
    origen text not null default 'importado' check (origen in ('manual','importado')),
    created_at timestamptz not null default now(),
    unique (estacion_id, anio, mes, producto_nombre_original)
);
comment on table public.sjap_sicom_mensual is 'Espeja la hoja "Sicom": informe regulatorio mensual de
↳ inventario por producto, con la fórmula de evaporación.';

do $$
declare
    t text;
begin
    for t in select unnest(array['sjap_urea_diario','sjap_presupuesto_mensual','sjap_sicom_mensual'])
    loop
        execute format('alter table public.%I enable row level security', t);
        execute format('create policy %I on public.%I for all to authenticated using (true) with check
↳ (true)', t || '_authenticated_all', t);
        execute format('create policy %I on public.%I for all to anon using (true) with check (true)', t ||
↳ '_anon_poc', t);
    end loop;
end $$;
;
```

```
-- -----
-- 20260910154227_create_sjap_promotores_e_islas.sql
-- -----

create table if not exists sjap_promotores (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  nombre text not null,
  activo boolean not null default true,
  created_at timestamptz not null default now(),
  unique (estacion_id, nombre)
);

create table if not exists sjap_islas (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  nombre text not null,
  activo boolean not null default true,
  created_at timestamptz not null default now(),
  unique (estacion_id, nombre)
);
alter table sjap_promotores enable row level security;
alter table sjap_islas enable row level security;

create policy sjap_promotores_authenticated_all on sjap_promotores for all to authenticated using (true)
  with check (true);
create policy sjap_promotores_anon_poc on sjap_promotores for all to anon using (true) with check (true);

create policy sjap_islas_authenticated_all on sjap_islas for all to authenticated using (true) with
  check (true);
create policy sjap_islas_anon_poc on sjap_islas for all to anon using (true) with check (true);

-- semilla desde los nombres que ya existen en el histórico, para no partir de cero
insert into sjap_promotores (estacion_id, nombre)
select distinct estacion_id, promotor_nombre
from sjap_ventas_promotor_turno
where promotor_nombre is not null
on conflict (estacion_id, nombre) do nothing;

insert into sjap_islas (estacion_id, nombre)
select distinct estacion_id, isla
from sjap_ventas_promotor_turno
where isla is not null and isla ~ '^[0-9]+$'
on conflict (estacion_id, nombre) do nothing;
;

-- -----
-- 20260910174944_create_turno_tipos_ausencias_programados.sql
-- -----

create table if not exists sjap_turno_tipos (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  nombre text not null,
  hora_inicio time not null,
  hora_fin time not null,
  orden int not null default 0,
  activo boolean not null default true,
  created_at timestamptz not null default now(),
  unique (estacion_id, nombre)
);
```

```

create table if not exists sjap_ausencias (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  promotor_id uuid not null references sjap_promotores(id),
  fecha_desde date not null,
  fecha_hasta date not null,
  tipo text not null check (tipo in ('vacaciones','incapacidad','permiso','ausencia','dia_libre')),
  nota text,
  created_at timestamptz not null default now(),
  check (fecha_hasta >= fecha_desde)
);

create table if not exists sjap_turnos_programados (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  fecha date not null,
  promotor_id uuid not null references sjap_promotores(id),
  isla_id uuid not null references sjap_islas(id),
  turno_tipo_id uuid not null references sjap_turno_tipos(id),
  estado text not null default 'programado' check (estado in ('programado','ausente','reemplazado')),
  created_at timestamptz not null default now()
);

create index if not exists idx_sjap_turnos_programados_fecha on sjap_turnos_programados (estacion_id,
  ↵ fecha);
create index if not exists idx_sjap_ausencias_rango on sjap_ausencias (estacion_id, promotor_id,
  ↵ fecha_desde, fecha_hasta);

alter table sjap_turno_tipos enable row level security;
alter table sjap_ausencias enable row level security;
alter table sjap_turnos_programados enable row level security;

create policy sjap_turno_tipos_authenticated_all on sjap_turno_tipos for all to authenticated using
  ↵ (true) with check (true);
create policy sjap_turno_tipos_anon_poc on sjap_turno_tipos for all to anon using (true) with check
  ↵ (true);

create policy sjap_ausencias_authenticated_all on sjap_ausencias for all to authenticated using (true)
  ↵ with check (true);
create policy sjap_ausencias_anon_poc on sjap_ausencias for all to anon using (true) with check (true);

create policy sjap_turnos_programados_authenticated_all on sjap_turnos_programados for all to
  ↵ authenticated using (true) with check (true);
create policy sjap_turnos_programados_anon_poc on sjap_turnos_programados for all to anon using (true)
  ↵ with check (true);

insert into sjap_turno_tipos (estacion_id, nombre, hora_inicio, hora_fin, orden)
select '884b3769-2c20-4a0b-8019-6972bf5e4caa', v.nombre, v.hora_inicio::time, v.hora_fin::time, v.orden
from (values
  ('T1','06:00','12:00',1),
  ('T2','12:00','18:00',2),
  ('T3','18:00','23:59',3),
  ('T4','00:00','06:00',4)
) as v(nombre, hora_inicio, hora_fin, orden)
on conflict (estacion_id, nombre) do nothing;
;

-----
-- 20260910181013_create_sjap_usuarios.sql

```

```

-----
create table if not exists sjap_usuarios (
  id uuid primary key default gen_random_uuid(),
  auth_user_id uuid not null unique references auth.users(id) on delete cascade,
  estacion_id uuid not null references sjap_estaciones(id),
  username text not null unique,
  rol text not null default 'dependiente' check (rol in ('master','dependiente')),
  created_at timestamptz not null default now()
);
alter table sjap_usuarios enable row level security;

create policy sjap_usuarios_authenticated_all on sjap_usuarios for all to authenticated using (true)
↳ with check (true);
create policy sjap_usuarios_anon_poc on sjap_usuarios for all to anon using (true) with check (true);
;

-----
-- 20260917212709_sjap_rls_estacion_scope.sql
-----
-- Helper: estación del usuario autenticado actual (SECURITY DEFINER para poder
-- leer sjap_usuarios sin depender de la política de esa misma tabla).
create or replace function sjap_estacion_de_usuario()
returns uuid
language sql
security definer
stable
set search_path = public
as $$
  select estacion_id from sjap_usuarios where auth_user_id = auth.uid() limit 1;
$$;

-- sjap_estaciones (tabla raíz: es la estación, no tiene estacion_id)
drop policy if exists sjap_estaciones_anon_poc on sjap_estaciones;
drop policy if exists sjap_estaciones_authenticated_all on sjap_estaciones;
create policy sjap_estaciones_scope on sjap_estaciones for all to authenticated
  using (id = sjap_estacion_de_usuario())
  with check (id = sjap_estacion_de_usuario());
-- Tablas con estacion_id directo
do $$
declare
  t text;
  tablas text[] := array[
    'sjap_productos','sjap_archivos_cierre','sjap_transacciones',
    'sjap_ventas_promotor_resumen','sjap_ventas_medio_pago','sjap_despachos_producto',
    'sjap_lecturas_inventario','sjap_facturas_compra','sjap_efectivo_diario',
    'sjap_cuentas_cliente','sjap_ventas_promotor_turno','sjap_balance_diario_producto',
    'sjap_cierre_diario','sjap_urea_diario','sjap_presupuesto_mensual','sjap_sicom_mensual',
    'sjap_promotores','sjap_islas','sjap_turno_tipos','sjap_ausencias',
    'sjap_turnos_programados','sjap_usuarios','sjap_auditoria'
  ];
begin
  foreach t in array tablas loop
    execute format('drop policy if exists %I on %I', t || '_anon_poc', t);
    execute format('drop policy if exists %I on %I', t || '_authenticated_all', t);
    execute format(
      'create policy %I on %I for all to authenticated using (estacion_id = sjap_estacion_de_usuario())
      ↳ with check (estacion_id = sjap_estacion_de_usuario())',

```

```

        t || '_scope', t
    );
end loop;
end $$;

-- Tablas escaladas via join (no tienen estacion_id propio)
drop policy if exists sjap_precios_producto_anon_poc on sjap_precios_producto;
drop policy if exists sjap_precios_producto_authenticated_all on sjap_precios_producto;
create policy sjap_precios_producto_scope on sjap_precios_producto for all to authenticated
    using (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
        ↳ p.estacion_id = sjap_estacion_de_usuario()))
    with check (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
        ↳ p.estacion_id = sjap_estacion_de_usuario()));

drop policy if exists sjap_archivos_cierre_historial_anon_poc on sjap_archivos_cierre_historial;
drop policy if exists sjap_archivos_cierre_historial_authenticated_all on sjap_archivos_cierre_historial;
create policy sjap_archivos_cierre_historial_scope on sjap_archivos_cierre_historial for all to
    ↳ authenticated
    using (exists (select 1 from sjap_archivos_cierre a where a.id =
        ↳ sjap_archivos_cierre_historial.archivo_id and a.estacion_id = sjap_estacion_de_usuario()))
    with check (exists (select 1 from sjap_archivos_cierre a where a.id =
        ↳ sjap_archivos_cierre_historial.archivo_id and a.estacion_id = sjap_estacion_de_usuario()));

drop policy if exists sjap_movimientos_cuenta_cliente_anon_poc on sjap_movimientos_cuenta_cliente;
drop policy if exists sjap_movimientos_cuenta_cliente_authenticated_all on
    ↳ sjap_movimientos_cuenta_cliente;
create policy sjap_movimientos_cuenta_cliente_scope on sjap_movimientos_cuenta_cliente for all to
    ↳ authenticated
    using (exists (select 1 from sjap_cuentas_cliente c where c.id =
        ↳ sjap_movimientos_cuenta_cliente.cuenta_id and c.estacion_id = sjap_estacion_de_usuario()))
    with check (exists (select 1 from sjap_cuentas_cliente c where c.id =
        ↳ sjap_movimientos_cuenta_cliente.cuenta_id and c.estacion_id = sjap_estacion_de_usuario()));
;

-----
-- 20260917212936_sjap_estacion_config.sql
-----

create table sjap_estacion_config (
    id uuid primary key default gen_random_uuid(),
    estacion_id uuid not null references sjap_estaciones(id),
    clave text not null,
    valor text not null,
    descripcion text,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique (estacion_id, clave)
);

alter table sjap_estacion_config enable row level security;
create policy sjap_estacion_config_scope on sjap_estacion_config for all to authenticated
    using (estacion_id = sjap_estacion_de_usuario())
    with check (estacion_id = sjap_estacion_de_usuario());

-- Semillas: los umbrales que hoy viven como literales dispersos en el código.
-- umbral_fluctuacion_galones: se documentaron dos valores distintos (5 y 100) para
-- la misma métrica en archivos distintos. Se elige 100 con base en los datos reales
-- de producción (fluctuaciones diarias de bioacem llegaron a ~1.700 galones en un
-- solo día en feb-2026; un corte de 5 marcaría como "peligroso" casi cualquier día).
insert into sjap_estacion_config (estacion_id, clave, valor, descripcion)

```

```

select id, 'umbral_fluctuacion_galones', '100', 'Fluctuación diaria de producto (galones) que se marca
↳ como alerta'
from sjap_estaciones
union all
select id, 'umbral_diferencia_caja_cop', '5000', 'Diferencia de caja (COP) que se marca como alerta'
from sjap_estaciones
union all
select id, 'umbral_sicom_galones', '50', 'Diferencia de inventario Sicom (galones) que se marca como
↳ alerta'
from sjap_estaciones
union all
select id, 'corte_cumplimiento_presupuesto', '1.0', 'Fracción de cumplimiento (compra/presupuesto) desde
↳ la cual se considera cumplido'
from sjap_estaciones;
;

```

```

-----
-- 20260917212954_sjap_medios_pago_catalogo.sql
-----

```

```

create table sjap_medios_pago (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  nombre text not null,
  orden smallint not null default 0,
  activo boolean not null default true,
  created_at timestamptz not null default now(),
  unique (estacion_id, nombre)
);
alter table sjap_medios_pago enable row level security;
create policy sjap_medios_pago_scope on sjap_medios_pago for all to authenticated
  using (estacion_id = sjap_estacion_de_usuario())
  with check (estacion_id = sjap_estacion_de_usuario());

-- Backfill: todos los medios de pago que ya existen en los datos reales,
-- incluyendo los que el arreglo hardcoded de CierreTab.jsx no contemplaba
-- (EFECTIVO, CREDITO, CREDITO CLIENTES, TARJETA DEBITO, TRANSFERENCIA).
insert into sjap_medios_pago (estacion_id, nombre, orden)
select e.id, m.medio_pago, row_number() over (order by m.medio_pago)
from sjap_estaciones e
cross join (select distinct medio_pago from sjap_ventas_medio_pago) m;
;

```

```

-----
-- 20260917212955_sjap_seed_medios_pago.sql
-----

```

```

-- Semilla del catálogo de medios de pago, para entornos NUEVOS.
--
-- Agregada el 2026-09-23. La migración 20260917212954 llena el catálogo copiando
-- los medios que ya existían en sjap_ventas_medio_pago; en una base vacía eso no
-- inserta nada. Esta lista reproduce el catálogo real de EDS LA FLORIDA.
-- Idempotente: en una base que ya tiene los medios no hace nada.

insert into public.sjap_medios_pago (estacion_id, nombre, orden)
select '884b3769-2c20-4a0b-8019-6972bf5e4caa', v.nombre, v.orden
from (values
  ('APP TERPEL', 1), ('BONO VIVE TERPEL', 2), ('CLIENTES PROPIOS', 3), ('CONSUMOS TERPEL', 4),
  ('CREDITO', 5), ('CREDITO CLIENTES', 6), ('DATAFONO', 7), ('EFECTIVO', 8), ('GOPASS', 9),
  ('MI EMPRESA', 10), ('QR', 11), ('RUMBO', 12), ('TARJETA DEBITO', 13), ('TRANSFERENCIA', 14),

```

```
( 'UREA', 15), ('UREA RUMBO', 16), ('VIVE TERPEL', 17)
) as v(nombre, orden)
where exists (select 1 from public.sjap_estaciones where id = '884b3769-2c20-4a0b-8019-6972bf5e4caa')
on conflict (estacion_id, nombre) do nothing;
;
```

```
-----
-- 20260917213002_sjap_ventas_promotor_turno_fk.sql
-----
```

```
alter table sjap_ventas_promotor_turno
  add column turno_tipo_id uuid references sjap_turno_tipos(id),
  add column promotor_id uuid references sjap_promotores(id);

-- Backfill turno: los 4 códigos T1-T4 coinciden 1:1 con sjap_turno_tipos.nombre.
update sjap_ventas_promotor_turno v
set turno_tipo_id = t.id
from sjap_turno_tipos t
where t.estacion_id = v.estacion_id and t.nombre = v.turno;
-- Backfill promotor: solo donde el nombre coincide exactamente con un promotor
-- individual del catálogo. Los nombres compuestos históricos ("ANGIE RUIZ/CAMILO
-- SANCHEZ", "SAMUEL MORENO/SANTIAGO LEON", "VALENTINA COSSIO /DANIEL ARGEL")
-- representan 2 personas en una sola fila del Excel original y deliberadamente
-- quedan sin promotor_id (ambigüedad real, no se fuerza un match incorrecto) -
-- promotor_nombre se conserva intacto para no perder esos datos históricos.
update sjap_ventas_promotor_turno v
set promotor_id = p.id
from sjap_promotores p
where p.estacion_id = v.estacion_id and p.nombre = v.promotor_nombre;
;
```

```
-----
-- 20260917213010_sjap_auditoria_estacion_not_null.sql
-----
```

```
alter table sjap_auditoria alter column estacion_id set not null;
;
```

```
-----
-- 20260917213204_sjap_presupuesto_producto_mensual.sql
-----
```

```
-- Presupuesto por producto, normalizado (reemplaza el supuesto de "siempre 3
-- productos fijos" que vivía como 3 columnas en sjap_presupuesto_mensual).
create table sjap_presupuesto_producto_mensual (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  producto_id uuid not null references sjap_productos(id),
  anio integer not null,
  mes integer not null check (mes >= 1 and mes <= 12),
  presupuesto numeric not null default 0,
  created_at timestamptz not null default now(),
  unique (estacion_id, producto_id, anio, mes)
);

alter table sjap_presupuesto_producto_mensual enable row level security;
create policy sjap_presupuesto_producto_mensual_scope on sjap_presupuesto_producto_mensual for all to
  ↳ authenticated
  using (estacion_id = sjap_estacion_de_usuario())
  with check (estacion_id = sjap_estacion_de_usuario());
```

```

-- Migra los datos existentes de las 3 columnas fijas a filas por producto.
-- Las columnas viejas se dejan intactas (no se rompe nada que las lea todavía),
-- simplemente dejan de ser la fuente de verdad hacia adelante.
insert into sjap_presupuesto_producto_mensual (estacion_id, producto_id, anio, mes, presupuesto)
select pm.estacion_id, p.id, pm.anio, pm.mes, pm.presupuesto_corriente
from sjap_presupuesto_mensual pm
join sjap_productos p on p.estacion_id = pm.estacion_id and p.codigo = 'CORRIENTE'
where pm.presupuesto_corriente is not null
union all
select pm.estacion_id, p.id, pm.anio, pm.mes, pm.presupuesto_acpm
from sjap_presupuesto_mensual pm
join sjap_productos p on p.estacion_id = pm.estacion_id and p.codigo = 'BIOACEM'
where pm.presupuesto_acpm is not null
union all
select pm.estacion_id, p.id, pm.anio, pm.mes, pm.presupuesto_extra
from sjap_presupuesto_mensual pm
join sjap_productos p on p.estacion_id = pm.estacion_id and p.codigo = 'EXTRA'
where pm.presupuesto_extra is not null;
;

-----
-- 20260917215852_sjap_estaciones_campos_config.sql
-----

alter table sjap_estaciones
  add column direccion text,
  add column telefono text,
  add column email text,
  add column moneda text not null default 'COP',
  add column zona_horaria text not null default 'America/Bogota';
;

-----
-- 20260917215933_sjap_rls_roles_config_master.sql
-----

create or replace function sjap_rol_de_usuario()
returns text
language sql
security definer
stable
set search_path = public
as $$
  select rol from sjap_usuarios where auth_user_id = auth.uid() limit 1;
$$;

do $$
declare
  t text;
  tablas_config text[] := array[
    'sjap_productos','sjap_medios_pago',
    'sjap_estacion_config','sjap_presupuesto_mensual','sjap_presupuesto_producto_mensual',
    'sjap_estaciones','sjap_usuarios'
  ];
begin
  foreach t in array tablas_config loop
    execute format('drop policy if exists %I on %I', t || '_scope', t);
    execute format(
      'create policy %I on %I for select to authenticated using (%s)',

```

```

        t || '_select', t,
        case when t in ('sjap_estaciones') then 'id = sjap_estacion_de_usuario()' else 'estacion_id =
↳ sjap_estacion_de_usuario()' end
    );
    execute format(
        'create policy %I on %I for insert to authenticated with check (%s and sjap_rol_de_usuario() =
↳ 'master')',
        t || '_write', t,
        case when t in ('sjap_estaciones') then 'id = sjap_estacion_de_usuario()' else 'estacion_id =
↳ sjap_estacion_de_usuario()' end
    );
    execute format(
        'create policy %I on %I for update to authenticated using (%s and sjap_rol_de_usuario() =
↳ 'master') with check (%s and sjap_rol_de_usuario() = 'master')',
        t || '_update', t,
        case when t in ('sjap_estaciones') then 'id = sjap_estacion_de_usuario()' else 'estacion_id =
↳ sjap_estacion_de_usuario()' end,
        case when t in ('sjap_estaciones') then 'id = sjap_estacion_de_usuario()' else 'estacion_id =
↳ sjap_estacion_de_usuario()' end
    );
    execute format(
        'create policy %I on %I for delete to authenticated using (%s and sjap_rol_de_usuario() =
↳ 'master')',
        t || '_delete', t,
        case when t in ('sjap_estaciones') then 'id = sjap_estacion_de_usuario()' else 'estacion_id =
↳ sjap_estacion_de_usuario()' end
    );
end loop;
end $$;
drop policy if exists sjap_precios_producto_scope on sjap_precios_producto;
create policy sjap_precios_producto_select on sjap_precios_producto for select to authenticated
using (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
↳ p.estacion_id = sjap_estacion_de_usuario()));
create policy sjap_precios_producto_write on sjap_precios_producto for insert to authenticated
with check (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
↳ p.estacion_id = sjap_estacion_de_usuario()) and sjap_rol_de_usuario() = 'master');
create policy sjap_precios_producto_update on sjap_precios_producto for update to authenticated
using (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
↳ p.estacion_id = sjap_estacion_de_usuario()) and sjap_rol_de_usuario() = 'master')
with check (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
↳ p.estacion_id = sjap_estacion_de_usuario()) and sjap_rol_de_usuario() = 'master');
create policy sjap_precios_producto_delete on sjap_precios_producto for delete to authenticated
using (exists (select 1 from sjap_productos p where p.id = sjap_precios_producto.producto_id and
↳ p.estacion_id = sjap_estacion_de_usuario()) and sjap_rol_de_usuario() = 'master');
;

-- -----
-- 20260917222050_sjap_estacion_config_freshness.sql
-- -----

insert into sjap_estacion_config (estacion_id, clave, valor, descripcion)
select id, 'data_freshness_warning_days', '3', 'Días sin cierre nuevo desde los que se muestra
↳ advertencia de datos desactualizados'
from sjap_estaciones
union all
select id, 'data_freshness_critical_days', '7', 'Días sin cierre nuevo desde los que se muestra alerta
↳ crítica de datos desactualizados'
from sjap_estaciones
on conflict (estacion_id, clave) do nothing;

```

```

;

-----
-- 20260923211204_sjap_esquemas_turno_y_reglas.sql
-----

-- 1. Esquemas de turno: un conjunto de turnos (2, 3, 4 o los que sean) que
-- se puede asignar por defecto, por día de la semana, por isla o por fecha.
create table sjap_esquemas_turno (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  nombre text not null,
  es_predeterminado boolean not null default false,
  created_at timestamptz not null default now(),
  unique (estacion_id, nombre)
);
create unique index sjap_esquemas_turno_un_predeterminado on sjap_esquemas_turno (estacion_id) where
  ↳ es_predeterminado;

alter table sjap_esquemas_turno enable row level security;
create policy sjap_esquemas_turno_scope on sjap_esquemas_turno for all to authenticated
  using (estacion_id = sjap_estacion_de_usuario())
  with check (estacion_id = sjap_estacion_de_usuario());

-- Los 4 turnos existentes pasan a un esquema predeterminado "Estándar":
-- ninguna asignación existente cambia de significado.
insert into sjap_esquemas_turno (estacion_id, nombre, es_predeterminado)
select id, 'Estándar', true from sjap_estaciones;

alter table sjap_turno_tipos add column esquema_id uuid references sjap_esquemas_turno(id);
update sjap_turno_tipos t set esquema_id = e.id
from sjap_esquemas_turno e where e.estacion_id = t.estacion_id and e.es_predeterminado;
alter table sjap_turno_tipos alter column esquema_id set not null;

-- El nombre deja de ser único por estación (dos esquemas pueden tener "Turno 1")
-- y pasa a ser único dentro del esquema. Diferible para permitir renombrar/
-- intercambiar nombres dentro de una misma transacción.
alter table sjap_turno_tipos drop constraint sjap_turno_tipos_estacion_id_nombre_key;
alter table sjap_turno_tipos add constraint sjap_turno_tipos_esquema_nombre_key
  unique (esquema_id, nombre) deferrable initially deferred;
-- 2. Reglas: qué esquema aplica. La más específica gana:
-- fecha+isla > fecha > día+isla > día > isla > esquema predeterminado.
create table sjap_esquema_reglas (
  id uuid primary key default gen_random_uuid(),
  estacion_id uuid not null references sjap_estaciones(id),
  esquema_id uuid not null references sjap_esquemas_turno(id) on delete cascade,
  isla_id uuid references sjap_islas(id),
  dia_semana smallint check (dia_semana between 0 and 6),
  fecha date,
  created_at timestamptz not null default now(),
  check (not (dia_semana is not null and fecha is not null)),
  check (isla_id is not null or dia_semana is not null or fecha is not null)
);
comment on column sjap_esquema_reglas.dia_semana is '0 = domingo ... 6 = sábado (misma convención que
  ↳ Date.getDay())';
create unique index sjap_esquema_reglas_un_alcance on sjap_esquema_reglas (
  estacion_id,
  coalesce(isla_id, '00000000-0000-0000-0000-000000000000'::uuid),
  coalesce(dia_semana, -1),

```

```

    coalesce(fecha, '0001-01-01'::date)
);

alter table sjap_esquema_reglas enable row level security;
create policy sjap_esquema_reglas_scope on sjap_esquema_reglas for all to authenticated
    using (estacion_id = sjap_estacion_de_usuario())
    with check (estacion_id = sjap_estacion_de_usuario());

-- 3. Validación: dentro de un esquema los turnos activos no pueden cruzarse
--    (incluidos los que pasan la medianoche) ni durar cero. Se evalúa al
--    cerrar la transacción, sobre el estado final.
create or replace function sjap_validar_turno_tipo() returns trigger
language plpgsql as $$
declare
    t record; o record;
    a1 int; a2 int; b1 int; b2 int;
begin
    select * into t from sjap_turno_tipos where id = new.id;
    if not found then return null; end if;
    a1 := extract(hour from t.hora_inicio)::int * 60 + extract(minute from t.hora_inicio)::int;
    a2 := extract(hour from t.hora_fin)::int * 60 + extract(minute from t.hora_fin)::int;
    if a1 = a2 then
        raise exception 'El turno "%" empieza y termina a la misma hora (%).', t.nombre,
        ↳ to_char(t.hora_inicio, 'HH24:MI');
    end if;
    if not t.activo then return null; end if;
    if a2 < a1 then a2 := a2 + 1440; end if;

    for o in select * from sjap_turno_tipos where esquema_id = t.esquema_id and activo and id <> t.id loop
        b1 := extract(hour from o.hora_inicio)::int * 60 + extract(minute from o.hora_inicio)::int;
        b2 := extract(hour from o.hora_fin)::int * 60 + extract(minute from o.hora_fin)::int;
        if b2 <= b1 then b2 := b2 + 1440; end if;
        if (a1 < b2 and b1 < a2) or (a1 < b2 + 1440 and b1 + 1440 < a2) or (a1 + 1440 < b2 and b1 < a2 +
        ↳ 1440) then
            raise exception 'El turno "%" (%-%) se cruza con "%" (%-%) en el mismo esquema.',
            t.nombre, to_char(t.hora_inicio, 'HH24:MI'), to_char(t.hora_fin, 'HH24:MI'),
            o.nombre, to_char(o.hora_inicio, 'HH24:MI'), to_char(o.hora_fin, 'HH24:MI');
        end if;
    end loop;
    return null;
end $$;

create constraint trigger sjap_turno_tipos_validar
after insert or update on sjap_turno_tipos
deferrable initially deferred
for each row execute function sjap_validar_turno_tipo();

-- 4. Guardar un esquema completo en una sola transacción. Corre con los
--    permisos del usuario (RLS aplica). Los turnos que se quitan del esquema se
--    borran si no tienen asignaciones; si tienen, se desactivan para no perder
--    historial.
create or replace function sjap_guardar_esquema_turnos(p_esquema_id uuid, p_turnos jsonb)
returns void
language plpgsql
set search_path = public
as $$
declare
    v_estacion uuid;
    item jsonb;

```

```

ids_conservados uuid[] := '{}';
v_id uuid;
i int := 0;
begin
select estacion_id into v_estacion from sjap_esquemas_turno where id = p_esquema_id;
if v_estacion is null then
    raise exception 'Esquema no encontrado.';
end if;

for item in select * from jsonb_array_elements(p_turnos) loop
    i := i + 1;
    if coalesce(trim(item->>'nombre'), '') = '' then
        raise exception 'Cada turno necesita un nombre.';
    end if;
    if nullif(item->>'id', '') is not null then
        update sjap_turno_tipos
            set nombre = trim(item->>'nombre'),
                hora_inicio = (item->>'hora_inicio')::time,
                hora_fin = (item->>'hora_fin')::time,
                orden = i,
                activo = true
            where id = (item->>'id')::uuid and esquema_id = p_esquema_id
            returning id into v_id;
        if v_id is null then
            raise exception 'Turno no encontrado en este esquema.';
        end if;
    else
        insert into sjap_turno_tipos (estacion_id, esquema_id, nombre, hora_inicio, hora_fin, orden,
↳ activo)
            values (v_estacion, p_esquema_id, trim(item->>'nombre'), (item->>'hora_inicio')::time,
↳ (item->>'hora_fin')::time, i, true)
            returning id into v_id;
        end if;
        ids_conservados := ids_conservados || v_id;
    end loop;

update sjap_turno_tipos set activo = false
    where esquema_id = p_esquema_id and not (id = any(ids_conservados))
    and exists (select 1 from sjap_turnos_programados p where p.turno_tipo_id = sjap_turno_tipos.id);
delete from sjap_turno_tipos
    where esquema_id = p_esquema_id and not (id = any(ids_conservados))
    and not exists (select 1 from sjap_turnos_programados p where p.turno_tipo_id =
↳ sjap_turno_tipos.id);
end $$;
;

-----
-- 20260926144116_sjap_auth_mejoras.sql
-----
-- Mejoras de autenticación y gestión de usuarios (2026-09-26).
--
-- 1. sjap_usuarios gana estado (activo), nombre visible y la marca
--    debe_cambiar_password (contraseña temporal asignada por un master).
-- 2. Un usuario desactivado deja de ver datos de inmediato: las funciones que
--    usan todas las políticas RLS solo reconocen usuarios activos. Además la
--    Edge Function sjap-usuarios lo bloquea en Supabase Auth (ban).
-- 3. Una estación nunca puede quedarse sin un master activo.
-- 4. Parámetro de cierre de sesión por inactividad.
-- 5. Endurecimiento señalado por los advisors de Supabase: las funciones de
--    rol/estación ya no se pueden ejecutar sin sesión, y el trigger de turnos

```

```
-- fija su search_path.

alter table public.sjap_usuarios
  add column if not exists activo boolean not null default true,
  add column if not exists nombre text,
  add column if not exists debe_cambiar_password boolean not null default false,
  add column if not exists updated_at timestampz not null default now();

comment on column public.sjap_usuarios.activo is 'false = usuario desactivado: no inicia sesión (ban en
↳ Auth) y las políticas RLS no le devuelven datos.';
comment on column public.sjap_usuarios.debe_cambiar_password is 'true cuando un master asignó una
↳ contraseña temporal: la app obliga a cambiarla al entrar.';

-- Solo usuarios ACTIVOS obtienen estación/rol → todas las políticas existentes
-- quedan cerradas para un usuario desactivado sin tener que tocarlas una a una.
create or replace function public.sjap_estacion_de_usuario()
returns uuid
language sql
security definer
stable
set search_path = public
as $$
  select estacion_id from public.sjap_usuarios where auth_user_id = auth.uid() and activo limit 1;
$$;

create or replace function public.sjap_rol_de_usuario()
returns text
language sql
security definer
stable
set search_path = public
as $$
  select rol from public.sjap_usuarios where auth_user_id = auth.uid() and activo limit 1;
$$;

revoke execute on function public.sjap_estacion_de_usuario() from public, anon;
revoke execute on function public.sjap_rol_de_usuario() from public, anon;
grant execute on function public.sjap_estacion_de_usuario() to authenticated, service_role;
grant execute on function public.sjap_rol_de_usuario() to authenticated, service_role;
alter function public.sjap_validar_turno_tipo() set search_path = public;

-- Nunca dejar una estación sin master activo (por cambio de rol, desactivación
-- o eliminación), venga el cambio de la app, del SQL Editor o de la API.
create or replace function public.sjap_proteger_ultimo_master()
returns trigger
language plpgsql
set search_path = public
as $$
begin
  if old.rol = 'master' and old.activo
    and (tg_op = 'DELETE' or new.rol <> 'master' or not new.activo) then
    if not exists (
      select 1 from public.sjap_usuarios
      where estacion_id = old.estacion_id and rol = 'master' and activo and id <> old.id
    ) then
      raise exception 'La estación debe conservar al menos un usuario master activo.'
      using errcode = 'P0001';
    end if;
  end if;
end if;
```

```

    if tg_op = 'DELETE' then
        return old;
    end if;
    new.updated_at := now();
    return new;
end;
$$;

drop trigger if exists sjap_usuarios_proteger_master on public.sjap_usuarios;
create trigger sjap_usuarios_proteger_master
    before update or delete on public.sjap_usuarios
    for each row execute function public.sjap_proteger_ultimo_master();

-- Minutos sin actividad antes de cerrar la sesión (equipos compartidos).
insert into public.sjap_estacion_config (estacion_id, clave, valor, descripcion)
select id, 'sesion_inactividad_minutos', '30', 'Minutos sin actividad antes de cerrar la sesión
    ↳ automáticamente (0 = nunca)'
from public.sjap_estaciones
on conflict (estacion_id, clave) do nothing;
;

-----
-- 20260927135046_sjap_permisos_por_rol.sql
-----

-- Integridad de datos y permisos por rol (2026-09-27). Parte 1 de 3: permisos.
--
-- Resultado de la auditoría de módulos:
-- * Casi todas las tablas permitían a cualquier usuario de la estación
--   (incluido un dependiente) insertar, editar y BORRAR por la API, incluso
--   catálogos, datos importados y la auditoría.
-- * Re-subir el archivo de un día borraba y recreaba el cierre, perdiendo el
--   efectivo real, la diferencia de caja y demás datos capturados o
--   importados, y no era transaccional.
-- * Guardar dos veces la lectura de tanque o el efectivo de un día fallaba
--   en silencio (restricción única) y un producto inactivo impedía cerrar.
-- * Los nombres únicos distinguían mayúsculas/espacios; los turnos
--   programados no tenían control de duplicados.
--
-- Matriz de permisos resultante (siempre limitada a la estación del usuario):
-- master      → todo.
-- dependiente → lee todo; opera el día (carga de archivos, insumos,
--                  facturas, movimientos, turnos y ausencias de hoy en
--                  adelante); no modifica catálogos, configuración, datos
--                  importados ni borra historia.

-----
-- 1. Funciones auxiliares
-----

create or replace function public.sjap_es_master()
returns boolean
language sql
stable
set search_path = public
as $$
    select coalesce(public.sjap_rol_de_usuario() = 'master', false);
$$;

-- Fecha de hoy en la zona horaria de la estación (por defecto Bogotá).
create or replace function public.sjap_hoy()

```

```

returns date
language sql
stable
set search_path = public
as $$
    select (now() at time zone coalesce(
        (select zona_horaria from public.sjap_estaciones where id = public.sjap_estacion_de_usuario()),
        'America/Bogota'))::date;
$$;
revoke execute on function public.sjap_es_master() from public, anon;
revoke execute on function public.sjap_hoy() from public, anon;
grant execute on function public.sjap_es_master() to authenticated, service_role;
grant execute on function public.sjap_hoy() to authenticated, service_role;

-- -----
-- 2. Políticas RLS por rol
-- -----

do $$
declare
    t text;
    alcance text;
    -- tabla → expresión de alcance por estación
    alcances jsonb := jsonb_build_object(
        'sjap_archivos_cierre_historial', 'exists (select 1 from sjap_archivos_cierre a where a.id =
        ↵ archivo_id and a.estacion_id = sjap_estacion_de_usuario())',
        'sjap_movimientos_cuenta_cliente', 'exists (select 1 from sjap_cuentas_cliente c where c.id =
        ↵ cuenta_id and c.estacion_id = sjap_estacion_de_usuario())'
    );
    -- catálogos, esquemas y datos importados sin captura en la app: solo master escribe
    solo_master text[] := array[
        'sjap_promotores', 'sjap_islas', 'sjap_esquemas_turno', 'sjap_turno_tipos', 'sjap_esquema_reglas',
        'sjap_cuentas_cliente', 'sjap_ventas_promotor_turno', 'sjap_sicom_mensual', 'sjap_urea_diario'];
    -- operación diaria: todos crean/editan, solo master borra
    operativas text[] := array[
        'sjap_archivos_cierre', 'sjap_transacciones', 'sjap_ventas_promotor_resumen', 'sjap_ventas_medio_pago',
        'sjap_despachos_producto', 'sjap_balance_diario_producto', 'sjap_cierre_diario',
        'sjap_lecturas_inventario', 'sjap_efectivo_diario'];
    -- bitácoras: solo se agregan filas, nadie las edita ni las borra
    bitacoras text[] := array['sjap_auditoria', 'sjap_archivos_cierre_historial'];
    -- registros manuales: todos crean; solo master corrige/borra y nunca lo importado
    manuales text[] := array['sjap_facturas_compra', 'sjap_movimientos_cuenta_cliente'];
    -- planificación: todos, con bloqueo de fechas pasadas por trigger
    planificacion text[] := array['sjap_turnos_programados', 'sjap_ausencias'];
    todas text[];
begin
    todas := solo_master || operativas || bitacoras || manuales || planificacion;
    foreach t in array todas loop
        alcance := coalesce(alcances->>t, 'estacion_id = sjap_estacion_de_usuario()');
        execute format('drop policy if exists %I on %I', t || '_scope', t);
        execute format('drop policy if exists %I on %I', t || '_select', t);
        execute format('drop policy if exists %I on %I', t || '_write', t);
        execute format('drop policy if exists %I on %I', t || '_update', t);
        execute format('drop policy if exists %I on %I', t || '_delete', t);
        execute format('create policy %I on %I for select to authenticated using (%s)', t || '_select', t,
        ↵ alcance);

        if t = any(solo_master) then
            execute format('create policy %I on %I for insert to authenticated with check ((%s) and

```

```

↳ sjap_es_master()), t || '_write', t, alcance);
    execute format('create policy %I on %I for update to authenticated using ((%)s) and
↳ sjap_es_master()) with check ((%)s) and sjap_es_master()), t || '_update', t, alcance, alcance);
    execute format('create policy %I on %I for delete to authenticated using ((%)s) and
↳ sjap_es_master()), t || '_delete', t, alcance);
    elsif t = any(operativas) then
        execute format('create policy %I on %I for insert to authenticated with check ((%)s)', t ||
↳ '_write', t, alcance);
        execute format('create policy %I on %I for update to authenticated using ((%)s) with check ((%)s)', t
↳ || '_update', t, alcance, alcance);
        execute format('create policy %I on %I for delete to authenticated using ((%)s) and
↳ sjap_es_master()), t || '_delete', t, alcance);
    elsif t = any(bitacoras) then
        execute format('create policy %I on %I for insert to authenticated with check ((%)s)', t ||
↳ '_write', t, alcance);
    elsif t = any(manuales) then
        execute format('create policy %I on %I for insert to authenticated with check ((%)s) and origen =
↳ 'manual'), t || '_write', t, alcance);
        execute format('create policy %I on %I for update to authenticated using ((%)s) and
↳ sjap_es_master() and origen = 'manual') with check ((%)s) and origen = 'manual'), t ||
↳ '_update', t, alcance, alcance);
        execute format('create policy %I on %I for delete to authenticated using ((%)s) and
↳ sjap_es_master() and origen = 'manual'), t || '_delete', t, alcance);
    else -- planificación
        execute format('create policy %I on %I for insert to authenticated with check ((%)s)', t ||
↳ '_write', t, alcance);
        execute format('create policy %I on %I for update to authenticated using ((%)s) with check ((%)s)', t
↳ || '_update', t, alcance, alcance);
        execute format('create policy %I on %I for delete to authenticated using ((%)s)', t || '_delete', t,
↳ alcance);
    end if;
end loop;
end $$;

```

--- 3. Nombres únicos sin distinguir mayúsculas/espacios y turnos sin duplicar

```

create unique index if not exists sjap_promotores_nombre_ci on public.sjap_promotores (estacion_id,
↳ lower(btrim(nombre)));
create unique index if not exists sjap_islas_nombre_ci on public.sjap_islas (estacion_id,
↳ lower(btrim(nombre)));
create unique index if not exists sjap_medios_pago_nombre_ci on public.sjap_medios_pago (estacion_id,
↳ lower(btrim(nombre)));
create unique index if not exists sjap_cuentas_cliente_nombre_ci on public.sjap_cuentas_cliente
↳ (estacion_id, lower(btrim(nombre)));
create unique index if not exists sjap_turnos_programados_unico
on public.sjap_turnos_programados (estacion_id, fecha, isla_id, turno_tipo_id, promotor_id);

alter table public.sjap_promotores add constraint sjap_promotores_nombre_no_vacio check (btrim(nombre)
↳ <> '') not valid;
alter table public.sjap_islas add constraint sjap_islas_nombre_no_vacio check (btrim(nombre) <> '') not
↳ valid;
alter table public.sjap_cuentas_cliente add constraint sjap_cuentas_cliente_nombre_no_vacio check
↳ (btrim(nombre) <> '') not valid;
alter table public.sjap_medios_pago add constraint sjap_medios_pago_nombre_no_vacio check (btrim(nombre)
↳ <> '') not valid;
alter table public.sjap_facturas_compra add constraint sjap_facturas_compra_cantidad_positiva check
↳ (cantidad > 0) not valid;
;

```

```
-- -----
-- 20260927135127_sjap_proteccion_historia.sql
-- -----
-- Integridad de datos (2026-09-27). Parte 2 de 3: protección de la historia.

-- -----
-- 4. Protección de la historia
-- -----
-- Turnos programados: los días anteriores a hoy quedan de solo lectura para
-- quien no es master. Sin sesión (SQL Editor / servicios) no aplica.
create or replace function public.sjap_turnos_proteger_pasado()
returns trigger
language plpgsql
set search_path = public
as $$
declare
    v_hoy date;
begin
    if auth.uid() is null or sjap_es_master() then
        return coalesce(new, old);
    end if;
    v_hoy := sjap_hoy();
    if (tg_op in ('UPDATE', 'DELETE') and old.fecha < v_hoy) or (tg_op in ('INSERT', 'UPDATE') and
    ↵ new.fecha < v_hoy) then
        raise exception 'Los turnos de días anteriores a hoy ya no se pueden modificar. Pídeselo a un
    ↵ usuario master.' using errcode = 'P0001';
    end if;
    return coalesce(new, old);
end;
$$;

drop trigger if exists sjap_turnos_programados_pasado on public.sjap_turnos_programados;
create trigger sjap_turnos_programados_pasado
before insert or update or delete on public.sjap_turnos_programados
for each row execute function public.sjap_turnos_proteger_pasado();

-- Ausencias ya terminadas: solo un master puede corregirlas o borrarlas.
create or replace function public.sjap_ausencias_proteger_pasado()
returns trigger
language plpgsql
set search_path = public
as $$
begin
    if auth.uid() is null or sjap_es_master() then
        return coalesce(new, old);
    end if;
    if old.fecha_hasta < sjap_hoy() then
        raise exception 'Esta ausencia ya terminó y forma parte del historial. Solo un master puede
    ↵ modificarla.' using errcode = 'P0001';
    end if;
    return coalesce(new, old);
end;
$$;

drop trigger if exists sjap_ausencias_pasado on public.sjap_ausencias;
create trigger sjap_ausencias_pasado
before update or delete on public.sjap_ausencias
for each row execute function public.sjap_ausencias_proteger_pasado();
```

```

-- Corregir los valores extraídos del archivo (venta, galones, clientes) es
-- solo para master; la carga de archivos lo hace a través de su función.
create or replace function public.sjap_cierre_proteger_correcciones()
returns trigger
language plpgsql
set search_path = public
as $$
begin
    if auth.uid() is null or sjap_es_master() or current_setting('sjap.origen', true) = 'carga' then
        return new;
    end if;
    if new.venta_total is distinct from old.venta_total
        or new.venta_galones_total is distinct from old.venta_galones_total
        or new.numero_clientes is distinct from old.numero_clientes
        or new.fecha is distinct from old.fecha
        or new.estacion_id is distinct from old.estacion_id then
        raise exception 'Solo un usuario master puede corregir la venta, los galones o el número de clientes
↳ de un cierre.' using errcode = 'P0001';
    end if;
    return new;
end;
$$;

drop trigger if exists sjap_cierre_diario_correcciones on public.sjap_cierre_diario;
create trigger sjap_cierre_diario_correcciones
before update on public.sjap_cierre_diario
for each row execute function public.sjap_cierre_proteger_correcciones();

-- Catálogos referenciados por TEXTO en datos históricos: renombrar o borrar
-- rompería el cruce (ventas_medio_pago.medio_pago, ventas_promotor_turno.isla).
create or replace function public.sjap_uso_texto_isla(p_estacion uuid, p_nombre text)
returns bigint
language sql
stable
set search_path = public
as $$
    select count(*) from public.sjap_ventas_promotor_turno v
    where v.estacion_id = p_estacion
        and (upper(btrim(v.isla)) = upper(btrim(p_nombre))
            or upper(btrim(p_nombre)) = any (regexp_split_to_array(upper(btrim(v.isla)),
↳ '\s*(?:\sY\s|,|/|-|&)\s*')));
$$;

create or replace function public.sjap_catalogo_proteger_texto()
returns trigger
language plpgsql
set search_path = public
as $$
declare
    v_uso bigint;
begin
    if tg_op = 'UPDATE' and lower(btrim(new.nombre)) = lower(btrim(old.nombre)) then
        return new;
    end if;
    if tg_table_name = 'sjap_medios_pago' then
        select count(*) into v_uso from public.sjap_ventas_medio_pago
        where estacion_id = old.estacion_id and upper(btrim(medio_pago)) = upper(btrim(old.nombre));
    else
        v_uso := public.sjap_uso_texto_isla(old.estacion_id, old.nombre);
    end if;
end;

```

```

end if;
if v_uso > 0 then
    raise exception '"%" aparece en % registro(s) históricos con ese nombre: no se puede % sin romper
    ↳ los reportes. Desactívalo en su lugar.',
        old.nombre, v_uso, case when tg_op = 'DELETE' then 'eliminar' else 'renombrar' end
    using errcode = 'P0001';
end if;
return coalesce(new, old);
end;
$$;
drop trigger if exists sjap_medios_pago_texto on public.sjap_medios_pago;
create trigger sjap_medios_pago_texto
    before update of nombre or delete on public.sjap_medios_pago
    for each row execute function public.sjap_catalogo_proteger_texto();

drop trigger if exists sjap_islas_texto on public.sjap_islas;
create trigger sjap_islas_texto
    before update of nombre or delete on public.sjap_islas
    for each row execute function public.sjap_catalogo_proteger_texto();
-- Productos y cuentas de cliente borran en cascada su historia (balances,
-- facturas, lecturas, precios, movimientos): se impide borrarlos si la tienen.
create or replace function public.sjap_proteger_borrado_con_historia()
returns trigger
language plpgsql
set search_path = public
as $$
declare
    v_tiene boolean;
begin
    if tg_table_name = 'sjap_productos' then
        v_tiene := exists (select 1 from public.sjap_balance_diario_producto where producto_id = old.id)
            or exists (select 1 from public.sjap_facturas_compra where producto_id = old.id)
            or exists (select 1 from public.sjap_lecturas_inventario where producto_id = old.id)
            or exists (select 1 from public.sjap_despachos_producto where producto_id = old.id);
    else
        v_tiene := exists (select 1 from public.sjap_movimientos_cuenta_cliente where cuenta_id = old.id);
    end if;
    if v_tiene then
        raise exception '"%" tiene historial asociado y no se puede eliminar. Desactívalo en su lugar.',
            coalesce(to_jsonb(old)->>'nombre_visible', to_jsonb(old)->>'nombre') using errcode = 'P0001';
    end if;
    return old;
end;
$$;

drop trigger if exists sjap_productos_borrado on public.sjap_productos;
create trigger sjap_productos_borrado
    before delete on public.sjap_productos
    for each row execute function public.sjap_proteger_borrado_con_historia();

drop trigger if exists sjap_cuentas_cliente_borrado on public.sjap_cuentas_cliente;
create trigger sjap_cuentas_cliente_borrado
    before delete on public.sjap_cuentas_cliente
    for each row execute function public.sjap_proteger_borrado_con_historia();

```

```

-----
-- 5. Uso de cada elemento de catálogo (para decidir editar/borrar/desactivar)
-----

```

```

create or replace function public.sjap_uso_catalogos()
returns table (tipo text, id uuid, turnos bigint, ventas bigint, ausencias bigint, reglas bigint)
language sql
stable
set search_path = public
as $$
    select 'promotor', p.id,
        (select count(*) from sjap_turnos_programados t where t.promotor_id = p.id),
        (select count(*) from sjap_ventas_promotor_turno v where v.promotor_id = p.id),
        (select count(*) from sjap_ausencias a where a.promotor_id = p.id),
        0::bigint
    from sjap_promotores p where p.estacion_id = sjap_estacion_de_usuario()
    union all
    select 'isla', i.id,
        (select count(*) from sjap_turnos_programados t where t.isla_id = i.id),
        sjap_uso_texto_isla(i.estacion_id, i.nombre),
        0::bigint,
        (select count(*) from sjap_esquema_reglas r where r.isla_id = i.id)
    from sjap_islas i where i.estacion_id = sjap_estacion_de_usuario()
    union all
    select 'medio_pago', m.id, 0::bigint,
        (select count(*) from sjap_ventas_medio_pago v where v.estacion_id = m.estacion_id and
        upper(btrim(v.medio_pago)) = upper(btrim(m.nombre))),
        0::bigint, 0::bigint
    from sjap_medios_pago m where m.estacion_id = sjap_estacion_de_usuario();
$$;

```

```

revoke execute on function public.sjap_uso_catalogos() from public, anon;
grant execute on function public.sjap_uso_catalogos() to authenticated;

```

```

-----
-- 6. Esquemas de turno: un turno usado en ventas históricas se desactiva, no
--    se borra (antes fallaba por la llave foránea).
-----

```

```

create or replace function public.sjap_guardar_esquema_turnos(p_esquema_id uuid, p_turnos jsonb)
returns void
language plpgsql
set search_path to 'public'
as $function$
declare
    v_estacion uuid;
    item jsonb;
    ids_conservados uuid[] := '{}';
    v_id uuid;
    i int := 0;
begin
    select estacion_id into v_estacion from sjap_esquemas_turno where id = p_esquema_id;
    if v_estacion is null then
        raise exception 'Esquema no encontrado.';
    end if;

    for item in select * from jsonb_array_elements(p_turnos) loop
        i := i + 1;
        if coalesce(trim(item->>'nombre'), '') = '' then
            raise exception 'Cada turno necesita un nombre.';
        end if;
        if nullif(item->>'id', '') is not null then
            update sjap_turno_tipos

```

```

        set nombre = trim(item->>'nombre'),
            hora_inicio = (item->>'hora_inicio')::time,
            hora_fin = (item->>'hora_fin')::time,
            orden = i,
            activo = true
        where id = (item->>'id')::uuid and esquema_id = p_esquema_id
        returning id into v_id;
    if v_id is null then
        raise exception 'Turno no encontrado en este esquema.';
    end if;
else
    insert into sjap_turno_tipos (estacion_id, esquema_id, nombre, hora_inicio, hora_fin, orden,
↳ activo)
        values (v_estacion, p_esquema_id, trim(item->>'nombre'), (item->>'hora_inicio')::time,
↳ (item->>'hora_fin')::time, i, true)
        returning id into v_id;
    end if;
    ids_conservados := ids_conservados || v_id;
end loop;
update sjap_turno_tipos set activo = false
    where esquema_id = p_esquema_id and not (id = any(ids_conservados))
        and (exists (select 1 from sjap_turnos_programados p where p.turno_tipo_id = sjap_turno_tipos.id)
            or exists (select 1 from sjap_ventas_promotor_turno v where v.turno_tipo_id =
↳ sjap_turno_tipos.id));
delete from sjap_turno_tipos
    where esquema_id = p_esquema_id and not (id = any(ids_conservados))
        and not exists (select 1 from sjap_turnos_programados p where p.turno_tipo_id =
↳ sjap_turno_tipos.id)
        and not exists (select 1 from sjap_ventas_promotor_turno v where v.turno_tipo_id =
↳ sjap_turno_tipos.id);
end $function$;

-- Marcar un esquema como predeterminado en una sola operación (antes eran dos
-- updates desde el navegador y el primero filtraba por una estación indefinida).
create or replace function public.sjap_esquema_predeterminado(p_esquema_id uuid)
returns void
language plpgsql
set search_path = public
as $$
declare
    v_estacion uuid;
begin
    select estacion_id into v_estacion from sjap_esquemas_turno where id = p_esquema_id;
    if v_estacion is null then
        raise exception 'Esquema no encontrado.';
    end if;
    update sjap_esquemas_turno set es_predeterminado = false where estacion_id = v_estacion and
↳ es_predeterminado and id <> p_esquema_id;
    update sjap_esquemas_turno set es_predeterminado = true where id = p_esquema_id;
end;
$$;

revoke execute on function public.sjap_esquema_predeterminado(uuid) from public, anon;
grant execute on function public.sjap_esquema_predeterminado(uuid) to authenticated;
;

-----
-- 20260927135220_sjap_carga_e_insumos_transaccionales.sql
-----

```

```

-- Integridad de datos (2026-09-27). Parte 3 de 3: carga del archivo e insumos.

-----
-- 7. Carga del archivo de cierre en UNA transacción
-----
-- Reemplaza la secuencia de ~10 llamadas del navegador. Al reprocesar un día:
-- * reemplaza solo lo que viene del archivo (transacciones, resúmenes,
--   medios de pago, despachos, venta y galones del cierre);
-- * CONSERVA lo capturado o importado: efectivo real, diferencia de caja,
--   certificación, número de clientes, estado, recibos y lecturas reales.
-- SECURITY DEFINER para que un dependiente pueda reprocesar sin tener permiso
-- de borrar historia; la estación siempre es la del usuario que llama.

create or replace function public.sjap_procesar_archivo_cierre(
    p_fecha date,
    p_archivo jsonb,
    p_transacciones jsonb,
    p_ventas_promotor jsonb,
    p_ventas_medio_pago jsonb,
    p_despachos jsonb,
    p_advertencias jsonb default '[]'::jsonb
)
returns jsonb
language plpgsql
security definer
set search_path = public
as $$
declare
    v_estacion uuid := sjap_estacion_de_usuario();
    v_usuario text;
    v_archivo_id uuid;
    v_version int := 1;
    v_existia boolean := false;
    v_sin_resolver text[] := '{}';
    v_venta numeric;
    v_galones numeric;
    r record;
    v_precio numeric;
    v_ini numeric;
    v_recibos numeric;
    v_real numeric;
    v_teorico numeric;
    v_fluct numeric;
    v_acum_prev numeric;
    v_acum numeric;
begin
    if v_estacion is null then
        raise exception 'Tu usuario no tiene acceso a SJAP Balance.' using errcode = '42501';
    end if;
    if p_fecha is null then
        raise exception 'El archivo no trae una fecha reconocible en sus transacciones.';
    end if;
    select username into v_usuario from sjap_usuarios where auth_user_id = auth.uid();
    perform set_config('sjap.origen', 'carga', true);

    -- 1) archivo del día (uno por estación y fecha)
    select id, version into v_archivo_id, v_version
        from sjap_archivos_cierre where estacion_id = v_estacion and fecha = p_fecha for update;
    if found then
        v_existia := true;

```

```

v_version := v_version + 1;
delete from sjap_transacciones where archivo_id = v_archivo_id;
delete from sjap_ventas_promotor_resumen where archivo_id = v_archivo_id;
delete from sjap_ventas_medio_pago where archivo_id = v_archivo_id;
delete from sjap_despachos_producto where archivo_id = v_archivo_id;
update sjap_archivos_cierre set
    nombre_archivo = coalesce(p_archivo->>'nombre_archivo', nombre_archivo),
    tamano_bytes = (p_archivo->>'tamano_bytes')::bigint,
    hash_archivo = p_archivo->>'hash_archivo',
    estado = 'procesando', version = v_version, procesado_en = null,
    subido_por = v_usuario
where id = v_archivo_id;
else
    v_version := 1;
    insert into sjap_archivos_cierre (estacion_id, fecha, nombre_archivo, tamano_bytes, hash_archivo,
↳ estado, version, subido_por)
    values (v_estacion, p_fecha, coalesce(p_archivo->>'nombre_archivo', 'archivo.xlsx'),
↳ (p_archivo->>'tamano_bytes')::bigint,
        p_archivo->>'hash_archivo', 'procesando', 1, v_usuario)
    returning id into v_archivo_id;
end if;

-- 2) detalle y resúmenes tal como vienen en el archivo
insert into sjap_transacciones (archivo_id, estacion_id, categoria, consecutivo, prefijo, fecha, hora,
↳ promotor_nombre,
                                tipo_factura, producto_nombre, cantidad, unidad, subtotal, descuento,
↳ impuesto_total, total)
select v_archivo_id, v_estacion, x.categoria, x.consecutivo, x.prefijo, x.fecha, x.hora,
↳ x.promotor_nombre,
    x.tipo_factura, x.producto_nombre, x.cantidad, x.unidad, x.subtotal, coalesce(x.descuento, 0),
↳ coalesce(x.impuesto_total, 0), x.total
from jsonb_populate_recordset(null::sjap_transacciones, coalesce(p_transacciones, '[]'::jsonb)) x;

insert into sjap_ventas_promotor_resumen (archivo_id, estacion_id, fecha, promotor_nombre,
↳ numero_ventas, total_ventas)
select v_archivo_id, v_estacion, p_fecha, x.promotor_nombre, coalesce(x.numero_ventas, 0),
↳ coalesce(x.total_ventas, 0)
from jsonb_populate_recordset(null::sjap_ventas_promotor_resumen, coalesce(p_ventas_promotor,
↳ '[]'::jsonb)) x;
insert into sjap_ventas_medio_pago (archivo_id, estacion_id, fecha, medio_pago, numero_ventas,
↳ total_ventas)
select v_archivo_id, v_estacion, p_fecha, x.medio_pago, coalesce(x.numero_ventas, 0),
↳ coalesce(x.total_ventas, 0)
from jsonb_populate_recordset(null::sjap_ventas_medio_pago, coalesce(p_ventas_medio_pago,
↳ '[]'::jsonb)) x;

-- 3) despachos, resolviendo el producto por código o alias (sin mayúsculas)
insert into sjap_despachos_producto (archivo_id, estacion_id, producto_id, producto_nombre_original,
↳ fecha, unidad, cantidad, precio, descuento, venta_total)
select v_archivo_id, v_estacion,
    (select p.id from sjap_productos p
     where p.estacion_id = v_estacion
       and (upper(btrim(p.codigo)) = upper(btrim(x.producto_nombre_original))
            or upper(btrim(x.producto_nombre_original)) = any (select upper(btrim(a)) from
↳ unnest(p.alias) a))
    limit 1),
    x.producto_nombre_original, p_fecha, x.unidad, coalesce(x.cantidad, 0), x.precio,
↳ coalesce(x.descuento, 0), x.venta_total
from jsonb_populate_recordset(null::sjap_despachos_producto, coalesce(p_despachos, '[]'::jsonb)) x;

```

```

select coalesce(array_agg(distinct producto_nombre_original), '{}') into v_sin_resolver
  from sjap_despachos_producto where archivo_id = v_archivo_id and producto_id is null;

-- 4) balance por producto. Un producto puede venir en varias filas (cambio
--   de precio a mitad del día): se suman. Se conservan recibos y lectura
--   real ya registrados, y se recalcula la fluctuación con la fórmula del
--   balance de referencia (valor = fluctuación acumulada × precio).
--   Incluye los productos que ya tenían balance ese día y el archivo
--   nuevo no trae: quedan con ventas 0 en vez de conservar ventas viejas.
for r in
  select s.producto_id, sum(s.cantidad) as ventas
  from (
    select producto_id, cantidad from sjap_despachos_producto where archivo_id = v_archivo_id and
↳ producto_id is not null
    union all
    select producto_id, 0 from sjap_balance_diario_producto where estacion_id = v_estacion and fecha =
↳ p_fecha
  ) s
  group by s.producto_id
loop
  select precio into v_precio from sjap_precios_producto
    where producto_id = r.producto_id and vigente_desde <= p_fecha order by vigente_desde desc limit 1;

  -- inventario inicial = inventario final REAL del día anterior
  select coalesce(
    (select b.inventario_final_real from sjap_balance_diario_producto b
      where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha < p_fecha and
↳ b.inventario_final_real is not null
      order by b.fecha desc limit 1),
    (select l.inventario_final_real from sjap_lecturas_inventario l
      where l.estacion_id = v_estacion and l.producto_id = r.producto_id and l.fecha < p_fecha
      order by l.fecha desc limit 1),
    (select b.inventario_inicial from sjap_balance_diario_producto b
      where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha = p_fecha))
  into v_ini;

  select b.recibos_galones, b.inventario_final_real into v_recibos, v_real
    from sjap_balance_diario_producto b
    where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha = p_fecha;
  v_recibos := coalesce(v_recibos, 0);
  if v_real is null then
    select l.inventario_final_real into v_real from sjap_lecturas_inventario l
      where l.estacion_id = v_estacion and l.producto_id = r.producto_id and l.fecha = p_fecha;
  end if;

  v_teorico := case when v_ini is not null then v_ini + v_recibos - r.ventas end;
  v_fluct := case when v_real is not null and v_teorico is not null then v_real - v_teorico end;
  select b.fluctuacion_acumulada into v_acum_prev from sjap_balance_diario_producto b
    where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha < p_fecha and
↳ b.fluctuacion_acumulada is not null
    order by b.fecha desc limit 1;
  v_acum := case when v_fluct is not null then coalesce(v_acum_prev, 0) + v_fluct end;

  insert into sjap_balance_diario_producto as b (estacion_id, producto_id, fecha, inventario_inicial,
↳ ventas_galones, recibos_galones,
    inventario_teorico, inventario_final_real, fluctuacion_dia, fluctuacion_acumulada, precio_vigente,
↳ fluctuacion_valor, estado, calculado_en)
  values (v_estacion, r.producto_id, p_fecha, v_ini, r.ventas, v_recibos, v_teorico, v_real, v_fluct,
↳ v_acum, v_precio,

```

```

        case when v_acum is not null and v_precio is not null then v_acum * v_precio end,
        case when v_real is not null then 'completo' else 'pendiente_insumos' end, now())
on conflict (estacion_id, producto_id, fecha) do update set
    inventario_inicial = excluded.inventario_inicial,
    ventas_galones = excluded.ventas_galones,
    inventario_teorico = excluded.inventario_teorico,
    inventario_final_real = excluded.inventario_final_real,
    fluctuacion_dia = excluded.fluctuacion_dia,
    fluctuacion_acumulada = excluded.fluctuacion_acumulada,
    precio_vigente = excluded.precio_vigente,
    fluctuacion_valor = excluded.fluctuacion_valor,
    estado = excluded.estado,
    calculado_en = now();
end loop;

-- 5) cierre del día: solo se reemplaza lo que viene del archivo
select coalesce(sum(venta_total), 0), coalesce(sum(cantidad), 0) into v_venta, v_galones
from sjap_despachos_producto where archivo_id = v_archivo_id;

insert into sjap_cierre_diario (estacion_id, archivo_id, fecha, venta_total, venta_galones_total,
↳ estado, calculado_en)
values (v_estacion, v_archivo_id, p_fecha, v_venta, v_galones, 'pendiente_insumos', now())
on conflict (estacion_id, fecha) do update set
    archivo_id = excluded.archivo_id,
    venta_total = excluded.venta_total,
    venta_galones_total = excluded.venta_galones_total,
    calculado_en = now();

perform sjap_recalcular_estado_cierre(v_estacion, p_fecha);
-- 6) trazabilidad
update sjap_archivos_cierre set
    estado = case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0 then 'con_errores' else 'procesado' end,
    procesado_en = now(),
    errores = case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0
        then jsonb_build_object('advertencias', p_advertencias, 'sinResolver',
↳ to_jsonb(v_sin_resolver)) end
where id = v_archivo_id;
insert into sjap_archivos_cierre_historial (archivo_id, version, hash_archivo, estado_resultado,
↳ errores)
values (v_archivo_id, v_version, p_archivo->>'hash_archivo', 'ok',
    case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0
        then jsonb_build_object('advertencias', p_advertencias, 'sinResolver',
↳ to_jsonb(v_sin_resolver)) end);

insert into sjap_auditoria (estacion_id, archivo_id, entidad, entidad_id, accion, nivel, detalle,
↳ created_by)
values (v_estacion, v_archivo_id, 'sjap_archivos_cierre', v_archivo_id,
    case when v_existia then 'reprocesado' else 'procesado' end,
    case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0 then 'warning' else 'info' end,
    jsonb_build_object('transacciones', jsonb_array_length(coalesce(p_transacciones, '[]'::jsonb)),
        'advertenciasParser', p_advertencias, 'productosSinResolver',
↳ to_jsonb(v_sin_resolver), 'version', v_version),
    v_usuario);

```

```

    return jsonb_build_object('archivo_id', v_archivo_id, 'version', v_version, 'reprocesado', v_existia,
                              'productos_sin_resolver', to_jsonb(v_sin_resolver));
end;
$$;

-----
-- 8. Insumos manuales del día (lectura de tanque y efectivo) atómicos
-----

-- Completo = cada producto ACTIVO tiene lectura real ese día + efectivo real.
create or replace function public.sjap_recalcular_estado_cierre(p_estacion uuid, p_fecha date)
returns void
language plpgsql
set search_path = public
as $$
declare
    v_inventario boolean;
    v_efectivo boolean;
begin
    select bool_and(
        exists (select 1 from sjap_balance_diario_producto b where b.estacion_id = p_estacion and
        ↳ b.producto_id = p.id and b.fecha = p_fecha and b.inventario_final_real is not null)
        or exists (select 1 from sjap_lecturas_inventario l where l.estacion_id = p_estacion and
        ↳ l.producto_id = p.id and l.fecha = p_fecha))
    into v_inventario
    from sjap_productos p where p.estacion_id = p_estacion and p.activo;
    select efectivo_real is not null into v_efectivo from sjap_cierre_diario where estacion_id =
    ↳ p_estacion and fecha = p_fecha;
    update sjap_cierre_diario
        set estado = case when coalesce(v_inventario, false) and coalesce(v_efectivo, false) then 'completo'
        ↳ else 'pendiente_insumos' end
        where estacion_id = p_estacion and fecha = p_fecha and estado <> 'con_alertas';
end;
$$;

create or replace function public.sjap_registrar_inventario(p_fecha date, p_lecturas jsonb)
returns void
language plpgsql
set search_path = public
as $$
declare
    v_estacion uuid := sjap_estacion_de_usuario();
    v_usuario text;
    item jsonb;
    v_producto uuid;
    v_valor numeric;
    v_teorico numeric;
    v_precio numeric;
    v_fluct numeric;
    v_acum numeric;
begin
    if v_estacion is null then
        raise exception 'Tu usuario no tiene acceso a SJAP Balance.' using errcode = '42501';
    end if;
    select username into v_usuario from sjap_usuarios where auth_user_id = auth.uid();
    for item in select * from jsonb_array_elements(p_lecturas) loop
        v_producto := (item->>'producto_id')::uuid;
        v_valor := (item->>'valor')::numeric;
        if v_valor is null or v_valor < 0 then
            raise exception 'La lectura de inventario debe ser un número mayor o igual a cero.';

```

```

end if;
if not exists (select 1 from sjap_productos where id = v_producto and estacion_id = v_estacion) then
    raise exception 'Producto no encontrado en la estación.';
end if;

insert into sjap_lecturas_inventario (estacion_id, producto_id, fecha, inventario_final_real,
↳ origen, capturado_por, capturado_en)
values (v_estacion, v_producto, p_fecha, v_valor, 'manual', v_usuario, now())
on conflict (estacion_id, producto_id, fecha) do update
    set inventario_final_real = excluded.inventario_final_real, origen = 'manual',
        capturado_por = excluded.capturado_por, capturado_en = now();

select inventario_teorico, precio_vigente into v_teorico, v_precio
    from sjap_balance_diario_producto where estacion_id = v_estacion and producto_id = v_producto and
↳ fecha = p_fecha;
if found then
    v_fluct := case when v_teorico is not null then v_valor - v_teorico end;
    select coalesce(fluctuacion_acumulada, 0) into v_acum from sjap_balance_diario_producto
        where estacion_id = v_estacion and producto_id = v_producto and fecha < p_fecha and
↳ fluctuacion_acumulada is not null
        order by fecha desc limit 1;
    v_acum := case when v_fluct is not null then coalesce(v_acum, 0) + v_fluct end;
    update sjap_balance_diario_producto set
        inventario_final_real = v_valor,
        fluctuacion_dia = v_fluct,
        fluctuacion_acumulada = v_acum,
        fluctuacion_valor = case when v_acum is not null and v_precio is not null then v_acum * v_precio
↳ end,
        estado = 'completo',
        calculado_en = now()
        where estacion_id = v_estacion and producto_id = v_producto and fecha = p_fecha;
end if;
end loop;
perform sjap_recalcular_estado_cierre(v_estacion, p_fecha);
end;
$$;

create or replace function public.sjap_registrar_efectivo(p_fecha date, p_efectivo numeric)
returns void
language plpgsql
set search_path = public
as $$
declare
    v_estacion uuid := sjap_estacion_de_usuario();
    v_usuario text;
begin
    if v_estacion is null then
        raise exception 'Tu usuario no tiene acceso a SJAP Balance.' using errcode = '42501';
    end if;
    if p_efectivo is null or p_efectivo < 0 then
        raise exception 'El efectivo real debe ser un número mayor o igual a cero.';
    end if;
    if not exists (select 1 from sjap_cierre_diario where estacion_id = v_estacion and fecha = p_fecha)
↳ then
        raise exception 'Primero carga el archivo de cierre de ese día.';
    end if;
    select username into v_usuario from sjap_usuarios where auth_user_id = auth.uid();
    insert into sjap_efectivo_diario (estacion_id, fecha, efectivo_real, origen, capturado_por,
↳ capturado_en)
    values (v_estacion, p_fecha, p_efectivo, 'manual', v_usuario, now())

```

```

on conflict (estacion_id, fecha) do update
    set efectivo_real = excluded.efectivo_real, origen = 'manual', capturado_por =
↳ excluded.capturado_por, capturado_en = now();
update sjap_cierre_diario set efectivo_real = p_efectivo where estacion_id = v_estacion and fecha =
↳ p_fecha;
perform sjap_recalcular_estado_cierre(v_estacion, p_fecha);
end;
$$;

revoke execute on function public.sjap_procesar_archivo_cierre(date, jsonb, jsonb, jsonb, jsonb, jsonb,
↳ jsonb) from public, anon;
revoke execute on function public.sjap_recalcular_estado_cierre(uuid, date) from public, anon;
-- INVOKER: la RLS limita la actualización a la estación del usuario.
grant execute on function public.sjap_recalcular_estado_cierre(uuid, date) to authenticated;
revoke execute on function public.sjap_registrar_inventario(date, jsonb) from public, anon;
revoke execute on function public.sjap_registrar_efectivo(date, numeric) from public, anon;
revoke execute on function public.sjap_uso_texto_isla(uuid, text) from public, anon;
grant execute on function public.sjap_procesar_archivo_cierre(date, jsonb, jsonb, jsonb, jsonb, jsonb,
↳ jsonb) to authenticated;
grant execute on function public.sjap_registrar_inventario(date, jsonb) to authenticated;
grant execute on function public.sjap_registrar_efectivo(date, numeric) to authenticated;
grant execute on function public.sjap_uso_texto_isla(uuid, text) to authenticated;
revoke execute on function public.sjap_guardar_esquema_turnos(uuid, jsonb) from public, anon;
grant execute on function public.sjap_guardar_esquema_turnos(uuid, jsonb) to authenticated;
;

-- -----
-- 20260927135517_sjap_carga_respetar_cierres.sql
-- -----
-- Carga de archivo: nunca borrar lo que el archivo nuevo no reemplaza (2026-09-27).
--
-- Hallado al reprocesar el 12-feb con la versión anterior (y con el código
-- previo del navegador, que hacía lo mismo): los medios de pago que solo
-- existen en el balance mensual importado (RUMBO, DATAFONO, QR...) se borraban
-- porque compartían archivo con los del POS. Ahora:
-- * medios de pago y resumen por promotor se actualizan por nombre; lo que
--   el archivo no trae se conserva;
-- * si el día ya está CERRADO (completo) el archivo solo actualiza su
--   detalle: no cambia los totales ni el balance conciliados y devuelve la
--   diferencia entre el archivo y el cierre para que el usuario la vea.
create or replace function public.sjap_procesar_archivo_cierre(
    p_fecha date,
    p_archivo jsonb,
    p_transacciones jsonb,
    p_ventas_promotor jsonb,
    p_ventas_medio_pago jsonb,
    p_despachos jsonb,
    p_advertencias jsonb default '[]'::jsonb
)
returns jsonb
language plpgsql
security definer
set search_path = public
as $$
declare
    v_estacion uuid := sjap_estacion_de_usuario();
    v_usuario text;
    v_archivo_id uuid;
    v_version int := 1;

```

```

v_existia boolean := false;
v_sin_resolver text[] := '{}';
v_venta numeric;
v_galones numeric;
r record;
v_precio numeric;
v_ini numeric;
v_recibos numeric;
v_real numeric;
v_teorico numeric;
v_fluct numeric;
v_acum_prev numeric;
v_acum numeric;
v_cerrado boolean := false;
v_venta_actual numeric;
v_galones_actual numeric;
v_diferencias jsonb := null;
begin
  if v_estacion is null then
    raise exception 'Tu usuario no tiene acceso a SJAP Balance.' using errcode = '42501';
  end if;
  if p_fecha is null then
    raise exception 'El archivo no trae una fecha reconocible en sus transacciones.';
  end if;
  select username into v_usuario from sjap_usuarios where auth_user_id = auth.uid();
  perform set_config('sjap.origen', 'carga', true);

  -- 1) archivo del día (uno por estación y fecha)
  select id, version into v_archivo_id, v_version
    from sjap_archivos_cierre where estacion_id = v_estacion and fecha = p_fecha for update;
  if found then
    v_existia := true;
    v_version := v_version + 1;
    delete from sjap_transacciones where archivo_id = v_archivo_id;
    delete from sjap_despachos_producto where archivo_id = v_archivo_id;
    update sjap_archivos_cierre set
      nombre_archivo = coalesce(p_archivo->>'nombre_archivo', nombre_archivo),
      tamano_bytes = (p_archivo->>'tamano_bytes')::bigint,
      hash_archivo = p_archivo->>'hash_archivo',
      estado = 'procesando', version = v_version, procesado_en = null,
      subido_por = v_usuario
    where id = v_archivo_id;
  else
    v_version := 1;
    insert into sjap_archivos_cierre (estacion_id, fecha, nombre_archivo, tamano_bytes, hash_archivo,
↳ estado, version, subido_por)
      values (v_estacion, p_fecha, coalesce(p_archivo->>'nombre_archivo', 'archivo.xlsx'),
↳ (p_archivo->>'tamano_bytes')::bigint,
      p_archivo->>'hash_archivo', 'procesando', 1, v_usuario)
    returning id into v_archivo_id;
  end if;

  -- 2) detalle y resúmenes tal como vienen en el archivo
  insert into sjap_transacciones (archivo_id, estacion_id, categoria, consecutivo, prefijo, fecha, hora,
↳ promotor_nombre,
                                tipo_factura, producto_nombre, cantidad, unidad, subtotal, descuento,
↳ impuesto_total, total)
  select v_archivo_id, v_estacion, x.categoria, x.consecutivo, x.prefijo, x.fecha, x.hora,
↳ x.promotor_nombre,
      x.tipo_factura, x.producto_nombre, x.cantidad, x.unidad, x.subtotal, coalesce(x.descuento, 0),

```

```

↳ coalesce(x.impuesto_total, 0), x.total
from jsonb_populate_recordset(null::sjap_transacciones, coalesce(p_transacciones, '[]'::jsonb)) x;

insert into sjap_ventas_promotor_resumen (archivo_id, estacion_id, fecha, promotor_nombre,
↳ numero_ventas, total_ventas)
select v_archivo_id, v_estacion, p_fecha, x.promotor_nombre, coalesce(x.numero_ventas, 0),
↳ coalesce(x.total_ventas, 0)
from jsonb_populate_recordset(null::sjap_ventas_promotor_resumen, coalesce(p_ventas_promotor,
↳ '[]'::jsonb)) x
on conflict (archivo_id, promotor_nombre) do update
    set numero_ventas = excluded.numero_ventas, total_ventas = excluded.total_ventas;

insert into sjap_ventas_medio_pago (archivo_id, estacion_id, fecha, medio_pago, numero_ventas,
↳ total_ventas)
select v_archivo_id, v_estacion, p_fecha, x.medio_pago, coalesce(x.numero_ventas, 0),
↳ coalesce(x.total_ventas, 0)
from jsonb_populate_recordset(null::sjap_ventas_medio_pago, coalesce(p_ventas_medio_pago,
↳ '[]'::jsonb)) x
on conflict (archivo_id, medio_pago) do update
    set numero_ventas = excluded.numero_ventas, total_ventas = excluded.total_ventas;
-- 3) despachos, resolviendo el producto por código o alias (sin mayúsculas)
insert into sjap_despachos_producto (archivo_id, estacion_id, producto_id, producto_nombre_original,
↳ fecha, unidad, cantidad, precio, descuento, venta_total)
select v_archivo_id, v_estacion,
    (select p.id from sjap_productos p
     where p.estacion_id = v_estacion
       and (upper(btrim(p.codigo)) = upper(btrim(x.producto_nombre_original))
          or upper(btrim(x.producto_nombre_original)) = any (select upper(btrim(a)) from
↳ unnest(p.alias) a))
    limit 1),
    x.producto_nombre_original, p_fecha, x.unidad, coalesce(x.cantidad, 0), x.precio,
↳ coalesce(x.descuento, 0), x.venta_total
from jsonb_populate_recordset(null::sjap_despachos_producto, coalesce(p_despachos, '[]'::jsonb)) x;

select coalesce(array_agg(distinct producto_nombre_original), '{}') into v_sin_resolver
    from sjap_despachos_producto where archivo_id = v_archivo_id and producto_id is null;

-- Día ya cerrado (completo): el archivo solo actualiza su detalle. Los
-- totales y el balance conciliados no se tocan; se informa la diferencia.
select estado = 'completo', venta_total, venta_galones_total into v_cerrado, v_venta_actual,
↳ v_galones_actual
    from sjap_cierre_diario where estacion_id = v_estacion and fecha = p_fecha;
v_cerrado := coalesce(v_cerrado, false);
select coalesce(sum(venta_total), 0), coalesce(sum(cantidad), 0) into v_venta, v_galones
    from sjap_despachos_producto where archivo_id = v_archivo_id;
if v_cerrado and (v_venta is distinct from v_venta_actual or v_galones is distinct from
↳ v_galones_actual) then
    v_diferencias := jsonb_build_object('venta_archivo', v_venta, 'venta_cierre', v_venta_actual,
        'galones_archivo', v_galones, 'galones_cierre',
↳ v_galones_actual);
end if;

-- 4) balance por producto (solo si el día no está cerrado). Un producto puede venir en varias filas
↳ (cambio
-- de precio a mitad del día): se suman. Se conservan recibos y lectura
-- real ya registrados, y se recalcula la fluctuación con la fórmula del
-- balance de referencia (valor = fluctuación acumulada × precio).
-- Incluye los productos que ya tenían balance ese día y el archivo
-- nuevo no trae: quedan con ventas 0 en vez de conservar ventas viejas.

```

```

for r in
  select s.producto_id, sum(s.cantidad) as ventas
  from (
    select producto_id, cantidad from sjap_despachos_producto where archivo_id = v_archivo_id and
  ↳ producto_id is not null
    union all
    select producto_id, 0 from sjap_balance_diario_producto where estacion_id = v_estacion and fecha =
  ↳ p_fecha
  ) s
  where not v_cerrado
  group by s.producto_id
loop
  select precio into v_precio from sjap_precios_producto
  where producto_id = r.producto_id and vigente_desde <= p_fecha order by vigente_desde desc limit 1;

  -- inventario inicial = inventario final REAL del día anterior
  select coalesce(
    (select b.inventario_final_real from sjap_balance_diario_producto b
     where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha < p_fecha and
  ↳ b.inventario_final_real is not null
     order by b.fecha desc limit 1),
    (select l.inventario_final_real from sjap_lecturas_inventario l
     where l.estacion_id = v_estacion and l.producto_id = r.producto_id and l.fecha < p_fecha
     order by l.fecha desc limit 1),
    (select b.inventario_inicial from sjap_balance_diario_producto b
     where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha = p_fecha))
  into v_ini;

  select b.recibos_galones, b.inventario_final_real into v_recibos, v_real
  from sjap_balance_diario_producto b
  where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha = p_fecha;
  v_recibos := coalesce(v_recibos, 0);
  if v_real is null then
    select l.inventario_final_real into v_real from sjap_lecturas_inventario l
    where l.estacion_id = v_estacion and l.producto_id = r.producto_id and l.fecha = p_fecha;
  end if;

  v_teorico := case when v_ini is not null then v_ini + v_recibos - r.ventas end;
  v_fluct := case when v_real is not null and v_teorico is not null then v_real - v_teorico end;
  select b.fluctuacion_acumulada into v_acum_prev from sjap_balance_diario_producto b
  where b.estacion_id = v_estacion and b.producto_id = r.producto_id and b.fecha < p_fecha and
  ↳ b.fluctuacion_acumulada is not null
  order by b.fecha desc limit 1;
  v_acum := case when v_fluct is not null then coalesce(v_acum_prev, 0) + v_fluct end;

  insert into sjap_balance_diario_producto as b (estacion_id, producto_id, fecha, inventario_inicial,
  ↳ ventas_galones, recibos_galones,
    inventario_teorico, inventario_final_real, fluctuacion_dia, fluctuacion_acumulada, precio_vigente,
  ↳ fluctuacion_valor, estado, calculado_en)
  values (v_estacion, r.producto_id, p_fecha, v_ini, r.ventas, v_recibos, v_teorico, v_real, v_fluct,
  ↳ v_acum, v_precio,
    case when v_acum is not null and v_precio is not null then v_acum * v_precio end,
    case when v_real is not null then 'completo' else 'pendiente_insumos' end, now())
  on conflict (estacion_id, producto_id, fecha) do update set
    inventario_inicial = excluded.inventario_inicial,
    ventas_galones = excluded.ventas_galones,
    inventario_teorico = excluded.inventario_teorico,
    inventario_final_real = excluded.inventario_final_real,
    fluctuacion_dia = excluded.fluctuacion_dia,
    fluctuacion_acumulada = excluded.fluctuacion_acumulada,

```

```

    precio_vigente = excluded.precio_vigente,
    fluctuacion_valor = excluded.fluctuacion_valor,
    estado = excluded.estado,
    calculado_en = now();
end loop;

-- 5) cierre del día: solo se reemplaza lo que viene del archivo
insert into sjap_cierre_diario (estacion_id, archivo_id, fecha, venta_total, venta_galones_total,
↳ estado, calculado_en)
values (v_estacion, v_archivo_id, p_fecha, v_venta, v_galones, 'pendiente_insumos', now())
on conflict (estacion_id, fecha) do update set
    archivo_id = excluded.archivo_id,
    venta_total = case when v_cerrado then sjap_cierre_diario.venta_total else excluded.venta_total end,
    venta_galones_total = case when v_cerrado then sjap_cierre_diario.venta_galones_total else
↳ excluded.venta_galones_total end,
    calculado_en = now();

if not v_cerrado then
    perform sjap_recalcular_estado_cierre(v_estacion, p_fecha);
end if;

-- 6) trazabilidad
update sjap_archivos_cierre set
    estado = case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0 then 'con_errores' else 'procesado' end,
    procesado_en = now(),
    errores = case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0
        then jsonb_build_object('advertencias', p_advertencias, 'sinResolver',
↳ to_jsonb(v_sin_resolver)) end
where id = v_archivo_id;

insert into sjap_archivos_cierre_historial (archivo_id, version, hash_archivo, estado_resultado,
↳ errores)
values (v_archivo_id, v_version, p_archivo->>'hash_archivo', 'ok',
    case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0
        then jsonb_build_object('advertencias', p_advertencias, 'sinResolver',
↳ to_jsonb(v_sin_resolver)) end);
insert into sjap_auditoria (estacion_id, archivo_id, entidad, entidad_id, accion, nivel, detalle,
↳ created_by)
values (v_estacion, v_archivo_id, 'sjap_archivos_cierre', v_archivo_id,
    case when v_existia then 'reprocesado' else 'procesado' end,
    case when jsonb_array_length(coalesce(p_advertencias, '[]'::jsonb)) > 0 or
↳ cardinality(v_sin_resolver) > 0 then 'warning' else 'info' end,
    jsonb_build_object('transacciones', jsonb_array_length(coalesce(p_transacciones, '[]'::jsonb)),
        'advertenciasParser', p_advertencias, 'productosSinResolver',
↳ to_jsonb(v_sin_resolver), 'version', v_version,
        'diaCerrado', v_cerrado, 'diferencias', v_diferencias),
    v_usuario);

return jsonb_build_object('archivo_id', v_archivo_id, 'version', v_version, 'reprocesado', v_existia,
    'productos_sin_resolver', to_jsonb(v_sin_resolver),
    'dia_cerrado', v_cerrado, 'diferencias', v_diferencias);

end;
$$;

-----
-- 8. Insumos manuales del día (lectura de tanque y efectivo) atómicos

```

```

-----
-- Completo = cada producto ACTIVO tiene lectura real ese día + efectivo real.
create or replace function public.sjap_recalcular_estado_cierre(p_estacion uuid, p_fecha date)
returns void
language plpgsql
set search_path = public
as $$
declare
    v_inventario boolean;
    v_efectivo boolean;
begin
    select bool_and(
        exists (select 1 from sjap_balance_diario_producto b where b.estacion_id = p_estacion and
        ↳ b.producto_id = p.id and b.fecha = p_fecha and b.inventario_final_real is not null)
        or exists (select 1 from sjap_lecturas_inventario l where l.estacion_id = p_estacion and
        ↳ l.producto_id = p.id and l.fecha = p_fecha))
    into v_inventario
    from sjap_productos p where p.estacion_id = p_estacion and p.activo;
    select efectivo_real is not null into v_efectivo from sjap_cierre_diario where estacion_id =
    ↳ p_estacion and fecha = p_fecha;
    update sjap_cierre_diario
        set estado = case when coalesce(v_inventario, false) and coalesce(v_efectivo, false) then 'completo'
        ↳ else 'pendiente_insumos' end
        where estacion_id = p_estacion and fecha = p_fecha and estado <> 'con_alertas';
end;
$$;
;

-----
-- Permisos para la API (por si el proyecto se creó sin "exponer tablas nuevas").
-- La seguridad real la dan las reglas RLS de arriba: cada usuario solo ve los
-- datos de su estación.
-----

grant usage on schema public to anon, authenticated;
do $$
declare r record;
begin
    for r in select tablename from pg_tables where schemaname = 'public' and tablename like 'sjap\_%' loop
        execute format('grant select, insert, update, delete on public.%I to authenticated', r.tablename);
    end loop;
    -- Funciones: solo usuarios con sesión (nunca anon).
    for r in select p.oid::regprocedure as fn from pg_proc p join pg_namespace n on n.oid = p.pronamespace
        where n.nspname = 'public' and p.proname like 'sjap\_%' loop
        execute format('grant execute on function %s to authenticated', r.fn);
    end loop;
end $$;

-- Fin del script 01. Continúa con 02_crear_usuario_master.sql

```